

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_d9866476-800\agent-a202c785689d506a5.jsonl`  
- SHA-256: `c7288aa512c85d4d28dd75f9cad13e4839261885e3fe8a87ff9b9b88cca0c3a6`  
- Source modified: `2026-06-22T09:13:54+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d9866476-800`  
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:12:06.303Z)

You are mapping the badminton-highlight-indexer repo, checked out at origin/master under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw structured data, not prose for a human.

ADVERSARIALLY VERIFY this claim ? try to REFUTE it with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e; default to "refuted"/"uncertain" if the evidence isn't clearly there. Claim:
"At least one OWNED, minors-free, ~5-6 minute golden video with cached WASB results (trajectory or DB) exists locally so the real configured+checkpointed reel-count nightly runs fast (seconds-to-low-minutes) without a fresh GPU pass each night."

Context from the mapping phase (may be wrong ? re-check against code):

```
[  
  {  
    "summary": "\nMapped the reel-production path end-to-end in the badminton-highlight-indexer.  
The regression test signal is: **COUNT OF PLAY_SEGMENTS ROWS in the SQLite database per video  
after segmentation + filtering**. One stitched output reel (ONE .mp4 file per /api/compile call)  
is created from N filtered play_segments. \n\nThe PIPELINE FLOW:\n1. /api/process (or CLI  
--video-path) ? _execute_processing ? video_id = MD5(video_file)\n2. Segmenter.process_video ?  
detects rally intervals ? inserts rows into play_segments table\n3. /api/compile receives  
segment_ids ? queries play_segments ? applies stitching.min_shot_count filter ? calls  
VideoCompiler.compile_highlights with (start_time, end_time) pairs\n4. VideoCompiler trims each  
segment (with begin/end padding), re-encodes with optional watermark, ffmpeg concat demuxer  
stitches N trimmed clips ? ONE .mp4 reel output file\n\n**Regression signal:** `SELECT COUNT(*)  
FROM play_segments WHERE video_id = ?` after segmentation completes. This is stable,  
deterministic, and queryable headlessly via the database.\n",  
    "key_facts": [  
      "video_id = compute_video_id(local_path) = MD5 hash  
(backend/utls/hashing.py:compute_video_id)",  
      "play_segments table schema: id (AUTOINCREMENT), video_id, start_time, end_time, duration,  
server, receiver, metadata (JSON). Rows inserted by segmenter via db.add_play_segment().",  
      "Segmentation path: main.py::_execute_processing (line 439) ? segmenter.process_video() ?  
returns list of rally dicts ? each inserted as play_segments row via  
db_instance.add_play_segment()",  
      "Compilation path: /api/compile (line 705) ? query_play_segments(video_id, {}) ? filter by  
segment_ids + stitching.min_shot_count (from metadata['shot_count']) ? extract time_pairs  
[(start, end), ...] ? VideoCompiler.compile_highlights(video_path, time_pairs,  
output_filename)",  
      "VideoCompiler.compile_highlights (compiler.py line 305): normalizes timestamps ? merges  
overlapping ranges ? for each segment: _trim_one_segment() (line 192) ffmpeg -ss -to re-encodes  
with padding + watermark ? ffmpeg concat demuxer stitches clips ? ONE output .mp4 file",
```



```

drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 .github
-rw-r--r-- 1 avidu 197609  1117 Jun 22 14:36 .gitignore
-rw-r--r-- 1 avidu 197609  5525 Jun 22 14:36 CLAUDE.md
-rw-r--r-- 1 avidu 197609 35347 Jun 22 14:36 README.md
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 backend
-rw-r--r-- 1 avidu 197609  2735 Jun 22 14:36 config.json
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 deploy
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 docs
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 eval_baselines
-rw-r--r-- 1 avidu 197609  2344 Jun 22 14:36 mypy.ini
-rw-r--r-- 1 avidu 197609  4525 Jun 22 14:36 pyproject.toml
-rw-r--r-- 1 avidu 197609   245 Jun 22 14:36 pytest.ini
-rw-r--r-- 1 avidu 197609   642 Jun 22 14:36 requirements.txt
-rw-r--r-- 1 avidu 197609   884 Jun 22 14:36 ruff.toml
-rwxr-xr-x 1 avidu 197609  2055 Jun 22 14:36 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5620 Jun 22 14:36 run_eval.py
-rw-r--r-- 1 avidu 197609  4356 Jun 22 14:36 run_regression.py
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 scratch
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 scripts
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 tests
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 tools
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 training

```

4. assistant (2026-06-22T09:12:12.432Z)

Now let me check the eval_baselines directory and search for golden videos or cached data:

5. user (2026-06-22T09:12:14.389Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/fusion_golden.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/golden_fixtures.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/golden_real_fixtures.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/checkpoints/2026-06-eval-baseline
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/eval_baselines
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/nightly_baseline.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_fusion_golden_telemetry.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_golden_fixtures.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_golden_fixtures_split.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_golden_real_fixtures.py

```

6. assistant (2026-06-22T09:12:16.676Z)

Good, there are golden fixture files and baselines. Let me check these:

7. user (2026-06-22T09:12:17.738Z)

```

1      {
2      "doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
3      "configs": {
4          "baseline_off": {
5              "aggregate": {
6                  "dend_abs_median": 41.77921047714383,
7                  "dstart_abs_median": 42.467243955066174,

```

```
8         "f1@0.25": 0.39981006941991765,
9         "f1@0.5": 0.2366145803119009,
10        "merge_rate": 0.6380248744753217,
11        "precision": 0.2577557018251538,
12        "recall": 0.22719343607869802,
13        "segment_count_ratio": 0.5824947574930804,
14        "split_rate": 0.0,
15        "tol_f1": 0.1382064040388172,
16        "tol_precision": 0.14536686157033415,
17        "tol_recall": 0.13445461315285676
18    },
19    "iou": 0.5,
20    "min_crossings": 0,
21    "n": 15,
22    "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value
the milestone buys.",
23    "per_video": {
24        "Badminton_BXH_2": {
25            "f1@0.25": 0.0,
26            "f1@0.5": 0.0,
27            "merge_count": 3,
28            "merge_rate": 1.0,
29            "num_gt": 44,
30            "num_pred": 4,
31            "split_count": 0,
32            "tol_f1": 0.0
33        },
34        "Boxhill_Doubles": {
35            "f1@0.25": 0.10526315789473685,
36            "f1@0.5": 0.03508771929824561,
37            "merge_count": 7,
38            "merge_rate": 0.9534883720930233,
39            "num_gt": 43,
40            "num_pred": 14,
41            "split_count": 0,
42            "tol_f1": 0.0
43        },
44        "GX010128": {
45            "f1@0.25": 0.14925373134328357,
46            "f1@0.5": 0.05970149253731343,
47            "merge_count": 12,
48            "merge_rate": 0.9615384615384616,
49            "num_gt": 52,
50            "num_pred": 15,
51            "split_count": 0,
52            "tol_f1": 0.0
53        },
54        "GX010137": {
55            "f1@0.25": 0.9253731343283583,
56            "f1@0.5": 0.5970149253731343,
57            "merge_count": 3,
58            "merge_rate": 0.17647058823529413,
59            "num_gt": 34,
60            "num_pred": 33,
61            "split_count": 0,
62            "tol_f1": 0.4477611940298507
63        },
64        "GX010141": {
65            "f1@0.25": 0.5416666666666666,
66            "f1@0.5": 0.3333333333333333,
67            "merge_count": 5,
68            "merge_rate": 0.6551724137931034,
69            "num_gt": 29,
70            "num_pred": 19,
71            "split_count": 0,
```

```

72         "tol_f1": 0.3333333333333333
73     },
74     "GX020094": {
75         "f1@0.25": 0.7333333333333334,
76         "f1@0.5": 0.4444444444444445,
77         "merge_count": 3,
78         "merge_rate": 0.15,
79         "num_gt": 40,
80         "num_pred": 50,
81         "split_count": 0,
82         "tol_f1": 0.2888888888888889
83     },
84     "GX030094": {
85         "f1@0.25": 0.5352112676056339,
86         "f1@0.5": 0.28169014084507044,
87         "merge_count": 8,
88         "merge_rate": 0.5609756097560976,
89         "num_gt": 41,
90         "num_pred": 30,
91         "split_count": 0,
92         "tol_f1": 0.14084507042253522
93     },
94     "adarsh_avi_singles": {
95         "f1@0.25": 0.8601036269430052,
96         "f1@0.5": 0.6321243523316062,
97         "merge_count": 5,
98         "merge_rate": 0.10416666666666667,
99         "num_gt": 96,
100        "num_pred": 97,
101        "split_count": 0,
102        "tol_f1": 0.38341968911917096
103    },
104    "gbaaddy": {
105        "f1@0.25": 0.17142857142857143,
106        "f1@0.5": 0.08571428571428572,
107        "merge_count": 9,
108        "merge_rate": 0.8541666666666666,
109        "num_gt": 48,
110        "num_pred": 22,
111        "split_count": 0,
112        "tol_f1": 0.0
113    },
114    "kushagra_singles": {
115        "f1@0.25": 0.0,
116        "f1@0.5": 0.0,
117        "merge_count": 2,
118        "merge_rate": 1.0,
119        "num_gt": 32,
120        "num_pred": 2,
121        "split_count": 0,
122        "tol_f1": 0.0
123    },
124    "targetest_doubles": {
125        "f1@0.25": 0.8118811881188118,
126        "f1@0.5": 0.5346534653465348,
127        "merge_count": 4,
128        "merge_rate": 0.16326530612244897,
129        "num_gt": 49,
130        "num_pred": 52,
131        "split_count": 0,
132        "tol_f1": 0.297029702970297
133    },
134    "mahadevpura_1": {
135        "f1@0.25": 0.0,
136        "f1@0.5": 0.0,

```

```

137         "merge_count": 1,
138         "merge_rate": 1.0,
139         "num_gt": 10,
140         "num_pred": 1,
141         "split_count": 0,
142         "tol_f1": 0.0
143     },
144     "mahadevpura_2": {
145         "f1@0.25": 0.0,
146         "f1@0.5": 0.0,
147         "merge_count": 3,
148         "merge_rate": 0.975,
149         "num_gt": 40,
150         "num_pred": 5,
151         "split_count": 0,
152         "tol_f1": 0.0
153     },
154     "mahadevpura_singles": {
155         "f1@0.25": 0.6181818181818182,
156         "f1@0.5": 0.3636363636363636,
157         "merge_count": 6,
158         "merge_rate": 0.5161290322580645,
159         "num_gt": 31,
160         "num_pred": 24,
161         "split_count": 0,
162         "tol_f1": 0.1818181818181818
163     },
164     "testlarge_short": {
165         "f1@0.25": 0.5454545454545454,
166         "f1@0.5": 0.1818181818181818,
167         "merge_count": 1,
168         "merge_rate": 0.5,
169         "num_gt": 6,
170         "num_pred": 5,
171         "split_count": 0,
172         "tol_f1": 0.0
173     }
174 },
175 "preset": {
176     "blob_factor": 4.0,
177     "blob_fallback": false,
178     "hard_cap": false,
179     "inactivity_end": 0.0,
180     "inactivity_min_density": 0.34,
181     "inactivity_rally_thresh": 0.08,
182     "max_window_duration": 0.0,
183     "merge_gap": 2.0,
184     "min_window_duration": 2.0,
185     "name": "baseline_off",
186     "velocity_thresh": 0.02
187 },
188 },
189 "default": {
190     "aggregate": {
191         "dend_abs_median": 2.9720372594817066,
192         "dstart_abs_median": 3.277048437326215,
193         "f1@0.25": 0.6296769405815345,
194         "f1@0.5": 0.4681709925781376,
195         "merge_rate": 0.13955677464298155,
196         "precision": 0.39945838409296397,
197         "recall": 0.5917335962330964,
198         "segment_count_ratio": 1.4784540509868136,
199         "split_rate": 0.11823489673413166,
200         "tol_f1": 0.3532006050209596,
201         "tol_precision": 0.30232209566271007,

```

```

202         "tol_recall": 0.44486191614359216
203     },
204     "iou": 0.5,
205     "min_crossings": 0,
206     "n": 15,
207     "note": "SERVED production windowing \u2014 now the R1 milestone (cap=6 + R4)
after the #204 flip.",
208     "per_video": {
209         "Badminton_BXH_2": {
210             "f1@0.25": 0.6721311475409836,
211             "f1@0.5": 0.4426229508196722,
212             "merge_count": 2,
213             "merge_rate": 0.09090909090909091,
214             "num_gt": 44,
215             "num_pred": 78,
216             "split_count": 4,
217             "tol_f1": 0.3278688524590163
218         },
219         "Boxhill_Doubles": {
220             "f1@0.25": 0.5538461538461538,
221             "f1@0.5": 0.4307692307692308,
222             "merge_count": 0,
223             "merge_rate": 0.0,
224             "num_gt": 43,
225             "num_pred": 87,
226             "split_count": 9,
227             "tol_f1": 0.26153846153846155
228         },
229         "GX010128": {
230             "f1@0.25": 0.7218045112781956,
231             "f1@0.5": 0.6015037593984963,
232             "merge_count": 0,
233             "merge_rate": 0.0,
234             "num_gt": 52,
235             "num_pred": 81,
236             "split_count": 2,
237             "tol_f1": 0.46616541353383456
238         },
239         "GX010137": {
240             "f1@0.25": 0.9041095890410958,
241             "f1@0.5": 0.8219178082191781,
242             "merge_count": 0,
243             "merge_rate": 0.0,
244             "num_gt": 34,
245             "num_pred": 39,
246             "split_count": 1,
247             "tol_f1": 0.6849315068493151
248         },
249         "GX010141": {
250             "f1@0.25": 0.7450980392156864,
251             "f1@0.5": 0.5098039215686274,
252             "merge_count": 6,
253             "merge_rate": 0.4482758620689655,
254             "num_gt": 29,
255             "num_pred": 22,
256             "split_count": 0,
257             "tol_f1": 0.4313725490196078
258         },
259         "GX020094": {
260             "f1@0.25": 0.6666666666666665,
261             "f1@0.5": 0.4242424242424242,
262             "merge_count": 0,
263             "merge_rate": 0.0,
264             "num_gt": 40,
265             "num_pred": 59,

```

```

266         "split_count": 7,
267         "tol_f1": 0.36363636363636365
268     },
269     "GX030094": {
270         "f1@0.25": 0.6451612903225806,
271         "f1@0.5": 0.4677419354838709,
272         "merge_count": 0,
273         "merge_rate": 0.0,
274         "num_gt": 41,
275         "num_pred": 83,
276         "split_count": 6,
277         "tol_f1": 0.41935483870967744
278     },
279     "adarsh_avi_singles": {
280         "f1@0.25": 0.7873303167420814,
281         "f1@0.5": 0.6787330316742081,
282         "merge_count": 0,
283         "merge_rate": 0.0,
284         "num_gt": 96,
285         "num_pred": 125,
286         "split_count": 13,
287         "tol_f1": 0.5520361990950227
288     },
289     "gbaaddy": {
290         "f1@0.25": 0.6388888888888889,
291         "f1@0.5": 0.4166666666666667,
292         "merge_count": 1,
293         "merge_rate": 0.04166666666666664,
294         "num_gt": 48,
295         "num_pred": 96,
296         "split_count": 8,
297         "tol_f1": 0.2777777777777778
298     },
299     "kushagra_singles": {
300         "f1@0.25": 0.6753246753246752,
301         "f1@0.5": 0.41558441558441556,
302         "merge_count": 3,
303         "merge_rate": 0.1875,
304         "num_gt": 32,
305         "num_pred": 45,
306         "split_count": 6,
307         "tol_f1": 0.07792207792207793
308     },
309     "targetest_doubles": {
310         "f1@0.25": 0.8,
311         "f1@0.5": 0.6666666666666666,
312         "merge_count": 0,
313         "merge_rate": 0.0,
314         "num_gt": 49,
315         "num_pred": 71,
316         "split_count": 5,
317         "tol_f1": 0.5833333333333334
318     },
319     "mahadevpura_1": {
320         "f1@0.25": 0.0,
321         "f1@0.5": 0.0,
322         "merge_count": 2,
323         "merge_rate": 0.8,
324         "num_gt": 10,
325         "num_pred": 2,
326         "split_count": 0,
327         "tol_f1": 0.0
328     },
329     "mahadevpura_2": {
330         "f1@0.25": 0.4878048780487805,

```

```

331         "f1@0.5": 0.24390243902439024,
332         "merge_count": 9,
333         "merge_rate": 0.525,
334         "num_gt": 40,
335         "num_pred": 42,
336         "split_count": 4,
337         "tol_f1": 0.12195121951219512
338     },
339     "mahadevpura_singles": {
340         "f1@0.25": 0.7469879518072289,
341         "f1@0.5": 0.6024096385542168,
342         "merge_count": 0,
343         "merge_rate": 0.0,
344         "num_gt": 31,
345         "num_pred": 52,
346         "split_count": 7,
347         "tol_f1": 0.5301204819277108
348     },
349     "testlarge_short": {
350         "f1@0.25": 0.4,
351         "f1@0.5": 0.3,
352         "merge_count": 0,
353         "merge_rate": 0.0,
354         "num_gt": 6,
355         "num_pred": 14,
356         "split_count": 1,
357         "tol_f1": 0.2
358     }
359 },
360 "preset": {
361     "blob_factor": 4.0,
362     "blob_fallback": false,
363     "hard_cap": false,
364     "inactivity_end": 1.5,
365     "inactivity_min_density": 0.3,
366     "inactivity_rally_thresh": 0.2,
367     "max_window_duration": 6.0,
368     "merge_gap": 2.0,
369     "min_window_duration": 2.0,
370     "name": "served",
371     "velocity_thresh": 0.02
372 },
373 },
374 "milestone": {
375     "aggregate": {
376         "dend_abs_median": 2.9720372594817066,
377         "dstart_abs_median": 3.277048437326215,
378         "f1@0.25": 0.6296769405815345,
379         "f1@0.5": 0.4681709925781376,
380         "merge_rate": 0.13955677464298155,
381         "precision": 0.39945838409296397,
382         "recall": 0.5917335962330964,
383         "segment_count_ratio": 1.4784540509868136,
384         "split_rate": 0.11823489673413166,
385         "tol_f1": 0.3532006050209596,
386         "tol_precision": 0.30232209566271007,
387         "tol_recall": 0.44486191614359216
388     },
389     "iou": 0.5,
390     "min_crossings": 0,
391     "n": 15,
392     "note": "R3 cap=6 s + R4 rally-END inactivity trim \u2014 the shipped milestone
(== default post-#204).",
393     "per_video": {
394         "Badminton_BXH_2": {

```

```
395         "f1@0.25": 0.6721311475409836,
396         "f1@0.5": 0.4426229508196722,
397         "merge_count": 2,
398         "merge_rate": 0.09090909090909091,
399         "num_gt": 44,
400         "num_pred": 78,
401         "split_count": 4,
402         "tol_f1": 0.3278688524590163
403     },
404     "Boxhill_Doubles": {
405         "f1@0.25": 0.5538461538461538,
406         "f1@0.5": 0.4307692307692308,
407         "merge_count": 0,
408         "merge_rate": 0.0,
409         "num_gt": 43,
410         "num_pred": 87,
411         "split_count": 9,
412         "tol_f1": 0.26153846153846155
413     },
414     "GX010128": {
415         "f1@0.25": 0.7218045112781956,
416         "f1@0.5": 0.6015037593984963,
417         "merge_count": 0,
418         "merge_rate": 0.0,
419         "num_gt": 52,
420         "num_pred": 81,
421         "split_count": 2,
422         "tol_f1": 0.46616541353383456
423     },
424     "GX010137": {
425         "f1@0.25": 0.9041095890410958,
426         "f1@0.5": 0.8219178082191781,
427         "merge_count": 0,
428         "merge_rate": 0.0,
429         "num_gt": 34,
430         "num_pred": 39,
431         "split_count": 1,
432         "tol_f1": 0.6849315068493151
433     },
434     "GX010141": {
435         "f1@0.25": 0.7450980392156864,
436         "f1@0.5": 0.5098039215686274,
437         "merge_count": 6,
438         "merge_rate": 0.4482758620689655,
439         "num_gt": 29,
440         "num_pred": 22,
441         "split_count": 0,
442         "tol_f1": 0.4313725490196078
443     },
444     "GX020094": {
445         "f1@0.25": 0.6666666666666665,
446         "f1@0.5": 0.4242424242424242,
447         "merge_count": 0,
448         "merge_rate": 0.0,
449         "num_gt": 40,
450         "num_pred": 59,
451         "split_count": 7,
452         "tol_f1": 0.36363636363636365
453     },
454     "GX030094": {
455         "f1@0.25": 0.6451612903225806,
456         "f1@0.5": 0.4677419354838709,
457         "merge_count": 0,
458         "merge_rate": 0.0,
459         "num_gt": 41,
```

```
460         "num_pred": 83,
461         "split_count": 6,
462         "tol_f1": 0.41935483870967744
463     },
464     "adarsh_avi_singles": {
465         "f1@0.25": 0.7873303167420814,
466         "f1@0.5": 0.6787330316742081,
467         "merge_count": 0,
468         "merge_rate": 0.0,
469         "num_gt": 96,
470         "num_pred": 125,
471         "split_count": 13,
472         "tol_f1": 0.5520361990950227
473     },
474     "gbaaddy": {
475         "f1@0.25": 0.638888888888889,
476         "f1@0.5": 0.4166666666666667,
477         "merge_count": 1,
478         "merge_rate": 0.04166666666666664,
479         "num_gt": 48,
480         "num_pred": 96,
481         "split_count": 8,
482         "tol_f1": 0.2777777777777778
483     },
484     "kushagra_singles": {
485         "f1@0.25": 0.6753246753246752,
486         "f1@0.5": 0.41558441558441556,
487         "merge_count": 3,
488         "merge_rate": 0.1875,
489         "num_gt": 32,
490         "num_pred": 45,
491         "split_count": 6,
492         "tol_f1": 0.07792207792207793
493     },
494     "largetest_doubles": {
495         "f1@0.25": 0.8,
496         "f1@0.5": 0.6666666666666666,
497         "merge_count": 0,
498         "merge_rate": 0.0,
499         "num_gt": 49,
500         "num_pred": 71,
501         "split_count": 5,
502         "tol_f1": 0.5833333333333334
503     },
504     "mahadevpura_1": {
505         "f1@0.25": 0.0,
506         "f1@0.5": 0.0,
507         "merge_count": 2,
508         "merge_rate": 0.8,
509         "num_gt": 10,
510         "num_pred": 2,
511         "split_count": 0,
512         "tol_f1": 0.0
513     },
514     "mahadevpura_2": {
515         "f1@0.25": 0.4878048780487805,
516         "f1@0.5": 0.24390243902439024,
517         "merge_count": 9,
518         "merge_rate": 0.525,
519         "num_gt": 40,
520         "num_pred": 42,
521         "split_count": 4,
522         "tol_f1": 0.12195121951219512
523     },
524     "mahadevpura_singles": {
```

```

525         "f1@0.25": 0.7469879518072289,
526         "f1@0.5": 0.6024096385542168,
527         "merge_count": 0,
528         "merge_rate": 0.0,
529         "num_gt": 31,
530         "num_pred": 52,
531         "split_count": 7,
532         "tol_f1": 0.5301204819277108
533     },
534     "testlarge_short": {
535         "f1@0.25": 0.4,
536         "f1@0.5": 0.3,
537         "merge_count": 0,
538         "merge_rate": 0.0,
539         "num_gt": 6,
540         "num_pred": 14,
541         "split_count": 1,
542         "tol_f1": 0.2
543     }
544 },
545 "preset": {
546     "blob_factor": 4.0,
547     "blob_fallback": false,
548     "hard_cap": false,
549     "inactivity_end": 1.5,
550     "inactivity_min_density": 0.3,
551     "inactivity_rally_thresh": 0.2,
552     "max_window_duration": 6.0,
553     "merge_gap": 2.0,
554     "min_window_duration": 2.0,
555     "name": "milestone",
556     "velocity_thresh": 0.02
557 }
558 }
559 },
560 "generated_at": "2026-06-19",
561 "git_commit": "a240221",
562 "iou": 0.5,
563 "manifest": "output/human_lovo_manifest.json",
564 "tolerances": {
565     "metric_tol": 0.005,
566     "over_seg_tol": 0,
567     "rate_tol": 0.01
568 }
569 }
570

```

8. user (2026-06-22T09:12:18.346Z)

```

1      ""Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3      This module holds the real-fixture subsystem split out of ``golden_fixtures`` (which
keeps the
4      shared synthetic/replay primitives). It turns an OWNED, minors-free real trajectory +
golden labels
5      into a committable, de-attributed Tier-0 fixture, reusing M1's ``replay_fixture`` for
the snapshot.
6      (docs/GOLDEN_REGRESSION_FIXTURES.md §4c anti-attribution, §5 threat model, §7 row P1, §6
honest
7      limits.)
8
9      The shared primitives (constants, float/JSON helpers, ``replay_fixture``, the CSV
writers, the
10     manifest appender) live in ``golden_fixtures`` and are imported FROM it here ? there is

```

```

a single
11     source of truth, no duplicated logic/thresholds. ``golden_fixtures`` re-exports the
public names
12     defined here (``freeze_real_fixture``, ``RefusalError``, ``_scan_forbidden``) at the
bottom of that
13     module so every historical ``golden_fixtures.<name>`` import path stays valid.
14
15     §6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
16     are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
17     ""
18     from __future__ import annotations
19
20     import os
21     import shutil
22     import tempfile
23     from pathlib import Path
24     from typing import Any, Dict, List, Optional, Sequence, Tuple
25
26     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
27     from backend.eval.gt_loader import load_gt_intervals
28     from backend.eval.regression import gt_hash
29     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30
31     # --- shared primitives, imported FROM golden_fixtures (single source of truth)
-----
32     from backend.eval.golden_fixtures import (
33         CURRENT_SCHEMA,
34         FIXTURES_DIR,
35         FORBIDDEN_KEYS,
36         PINNED_MAX_WINDOW,
37         PINNED_MERGE_GAP,
38         PINNED_MIN_WINDOW,
39         PINNED_VT,
40         Interval,
41         _LABEL_SLUG_RE,
42         _MD5_RE,
43         _append_manifest_entry,
44         _round,
45         _write_json,
46         replay_fixture,
47         write_labels_csv,
48         write_trajectory_csv,
49     )
50
51
52     # =====
53     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
54     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
55     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
56     # =====
57
58
59     class RefusalError(ValueError):
60         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
61
62         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
it) that
63         names *why* the unsafe public-fixture path was refused. The refusal is the code-
enforced
64         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
minors-excluded
65         / license flags are human-asserted, but the gate makes the unsafe path impossible to

```

```

reach
66         *silently* ? a fixture cannot be written unless all three are affirmatively set.
67         ""
68
69
70     def _new_surrogate_slug() -> str:
71         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` ($4c).
72
73         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
random,
74         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
over a
75         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
76         ""
77         return f"fx_{os.urandom(8).hex()}"
78
79
80     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
path token
81     #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
regardless), so it is
82     #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
would let
83     #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
(red-team #4).
84     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
85         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
86         "provenance", "provenance.json",
87     )
88
89
90     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
91         """POSITIVE privacy assertion ($4c, red-team ?): raise if ``text`` leaks anything
attributable.
92
93         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
94         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
95         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
96         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
97         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
98         ""
99         low = text.lower()
100        for key in FORBIDDEN_KEYS:
101            if key in low:
102                raise RefusalError(
103                    f"{where}: forbidden key substring {key!r} found in written output ? "
104                    f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
105                    f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md $4c).")
106        m = _MD5_RE.search(text)
107        if m:
108            raise RefusalError(
109                f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
110                f"video_id shape and an attribution back-channel; never write it ($4c/$5).")
111        for sp in source_paths:
112            if not sp:
113                continue
114            p = Path(sp)

```

```

115         for token in (sp, p.name, p.stem):
116             t = (token or "").strip().lower()
117             # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
118             # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
119             # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
120             # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
121             # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
122             if not t or t in _SCHEMA_FILENAME_LITERALS:
123                 continue
124             if t in low:
125                 raise RefusalError(
126                     f"{where}: source-path token {token!r} found in written output ? the
source "
127                     f"filename/path must never be persisted ($4c metadata strip).")
128
129
130     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
131                               ) -> List[Tuple[int, int, float, float]]:
132         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
133
134         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
135         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
136         """
137         rows: List[Tuple[int, int, float, float]] = []
138         for p in points:
139             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
140         rows.sort(key=lambda r: r[0])
141         return rows
142
143
144     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
145         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant).
146
147         ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
Each label
148         endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
of two exact
149         6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
150         ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
reset score
151         delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
``freeze_real_fixture``
152         for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
second
153         offset (the grid, not a real metric divergence)."""
154         return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]
155
156
157     def freeze_real_fixture(
158         label: str,
159         trajectory_csv_path: Any,
160         labels_csv_path: Any,
161         *,
162         license: str,
163         minors_free: bool,
164         owned: bool,

```

```

165         fps: float,
166         frame_width: float,
167         fixtures_dir: Path = FIXTURES_DIR,
168         slug: Optional[str] = None,
169         time_origin_reset: bool = True,
170         source_note: str = "",
171     ) -> Dict[str, Any]:
172         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
173         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
row P1.)
174
175         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
176         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string,
177         AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text
identifier like a
178         player/event name is refused, red-team #1). The committed bytes carry NO filename /
video_id /
179         match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
180         shuttle-only trajectory (time-origin reset by default), the reset labels, and a
181         ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
182         ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.
183
184         ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
185         back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
186         contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
187         committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
188         retained for CLI compatibility / future logging.
189
190         NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
191         only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
192         attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
193         owner-asserted Fork-A / minors / biometric-exclusion flags.
194
195         The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
196         after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
197         (red-team #3).
198
199         Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
200         """
201         # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
202         reasons: List[str] = []
203         if minors_free is not True:
204             reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
205                             "of a minor is prohibited ? $3, guardrail 'exclude minors')")
206         if owned is not True:
207             reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; ")

```

```

208             "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
209         if not (isinstance(license, str) and license.strip()):
210             reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
211                             "'provenance hard-fail gate')")
212         # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
213         # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
214         if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
215             reasons.append(
216                 f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}
(lowercase "
217                 f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier
(player/event "
218                 f"name) is an attribution leak; got {label!r} (§4c metadata strip, red-team
#1)")
219         if reasons:
220             raise RefusalError(
221                 "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
222                 "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n - "
223                 + "\n - ".join(reasons))
224
225         traj_path = Path(str(trajjectory_csv_path))
226         labels_path = Path(str(labels_csv_path))
227
228         # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
229         out_slug = slug if slug is not None else _new_surrogate_slug()
230
231         # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
-----
232         points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
233         if not points:
234             raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
235         gts = load_gt_intervals(str(labels_path), strict=True)
236
237         # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, §4c) --
238         # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
239         # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
240         # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
241         # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
242         # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
243         # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
244         #
245         # HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while
labels shift
246         # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
247         # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
248         # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
249         # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).

```

```

250     if time_origin_reset:
251         t0_frame = min(int(p.frame) for p in points)
252         t0_sec = _round(t0_frame / float(fps))
253     else:
254         t0_frame, t0_sec = 0, 0.0
255     rows = _reset_trajectory_rows(points, t0_frame)
256     reset_gts = _reset_labels(gts, t0_sec)
257
258     # --- (d2) negative-label guard BEFORE any write (red-team #3a)
-----
259     # A reset label that starts < 0 means the label timeline predates the earliest
trajectory
260     # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
261     # start). Refuse loudly up-front so the partial-write path is never even reached.
262     neg = [(s, e) for (s, e) in reset_gts if s < 0]
263     if neg:
264         raise RefusalError(
265             f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "
266             f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
267             f"earliest label must not precede the earliest trajectory frame ($4c time-
origin reset).")
268
269     iou = 0.5
270     # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
271     # it never reaches the committed bytes (an identifying note is an attribution back-
channel
272     # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
273     prov = {
274         "slug": out_slug,
275         "schema_version": CURRENT_SCHEMA,
276         "kind": "cleared_real",
277         "label": str(label),
278         "license": license,
279         "minors_free": True,
280         "owned": True,
281         "fps": _round(fps),
282         "frame_width": _round(frame_width),
283         "iou": _round(iou),
284         "time_origin_reset": bool(time_origin_reset),
285         "windowing": {
286             "vt": PINNED_VT,
287             "mg": PINNED_MERGE_GAP,
288             "mwd": PINNED_MIN_WINDOW,
289             "max_win": PINNED_MAX_WINDOW,
290         },
291         "paths": {
292             "trajectory": "trajectory.csv",
293             "labels": "labels.csv",
294             "expected": "expected.json",
295         },
296         "gt_hash": gt_hash(reset_gts),
297         "note": (
298             "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
299             "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
300             "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
301             "Tier-0 / CONTROL-only. S6: behaviour-locking, not correctness; sits behind
counsel (P2).")

```

```

302         ),
303     }
304
305     # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
306     # Everything is written under a sibling temp dir; the final slug dir is created by
307     # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
308     # partial fixture dir behind. source_note (if any) is scanned defensively but never
309     # written.
310     fdir = fixtures_dir / out_slug
311     fixtures_dir.mkdir(parents=True, exist_ok=True)
312     if fdir.exists():
313         raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
314         (choose a new slug)")
315     tmp_parent = Path(tempfile.mkdtemp(prefix=f"{out_slug}.tmp.",
316     dir=str(fixtures_dir)))
317     tmp_dir = tmp_parent / out_slug
318     tmp_dir.mkdir(parents=True, exist_ok=True)
319     try:
320         write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
321         write_labels_csv(reset_gts, tmp_dir / "labels.csv")
322         _write_json(tmp_dir / "provenance.json", prov)
323
324         expected = replay_fixture(tmp_dir) # REUSE M1 ? correct-by-construction
325         snapshot
326         _write_json(tmp_dir / "expected.json", expected)
327
328         # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
329         source_note) ---
330         source_paths = [str(traj_path), str(labels_path)]
331         _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
332             where=f"{out_slug}/expected.json", source_paths=source_paths)
333         _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
334             where=f"{out_slug}/provenance.json", source_paths=source_paths)
335         # source_note is never persisted, but a leak in it is still a contract violation
336         ? scan it.
337         if source_note:
338             _scan_forbidden(str(source_note),
339                 where=f"{out_slug}/source_note(not-persisted)",
340                 source_paths=source_paths)
341
342         # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
343         -----
344         os.replace(str(tmp_dir), str(fdir))
345     finally:
346         # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
347         partial dir).
348         shutil.rmtree(str(tmp_parent), ignore_errors=True)
349
350     # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
351     -----
352     _append_manifest_entry(prov, fixtures_dir)
353     return expected
354

```

9. user (2026-06-22T09:12:19.313Z)

```

1     """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3     `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the **single shared
4     replay code path** used by both the freeze tool and the characterization gate, and it
5     deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6     variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip

```

```

7      across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8      only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9      merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10     LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11     `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12     path simply never calls into it.)
13
14     The replay reuses the already-shipped eval stack VERBATIM::
15
16         parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17         ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18         ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20     Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22     * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"``.
25     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27     * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28     passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29     named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30     silently churn fixtures.
31     * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32     the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33     takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34     has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35     ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36     tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38     §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39     suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42     field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44     Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45     parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48     never calls into it.

```

```

49     """
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import re
56     import sys
57     from dataclasses import dataclass
58     from pathlib import Path
59     from typing import Any, Dict, List, Optional, Sequence, Tuple
60
61     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
62     -----
63     from backend.eval import distill_local
64     from backend.eval.experiment import CONTROL, VideoCandidates, predict
65     from backend.eval.gt_loader import load_gt_intervals
66     from backend.eval.metrics import score
67     from backend.eval.regression import gt_hash
68
69     Interval = Tuple[float, float]
70
71     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
72     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
73     CURRENT_SCHEMA = 1
74
75     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
76     works
77     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
78     root.
79     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
80
81     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
82     merge_gap,
83     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
84     LITERAL
85     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
86     constant so a
87     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
88     fixture also
89     #: re-states these in its provenance; the replay reads them from there.
90     PINNED_VT = 0.02
91     PINNED_MERGE_GAP = 2.0
92     PINNED_MIN_WINDOW = 2.0
93     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
94     NOTE: this is
95     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
96     ``_pinned_proposal``
97     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
98     deliberately
99     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
100     the W1-OFF
101     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
102     wiring.
103     PINNED_MAX_WINDOW = 0.0
104
105     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
106     stated
107     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
108     exactly these.
109     SHUTTLE_FEATURES: Tuple[str, ...] = (
110         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
111     )
112     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
113         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
114     )

```

```

100     )
101
102     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
103     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
104     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
105
106     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
107     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
108
109     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
110     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
111     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
112
113     #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
114     #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
115     #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
116     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
117     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
118
119
120     # =====
121     # Synthetic trajectory generation (deterministic ? NO random)
122     # =====
123
124
125     @dataclass(frozen=True)
126     class RallySpec:
127         """One in-flight rally segment: the shuttle oscillates across the net midline.
128
129         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
130         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
131         net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
132         """
133
134         start_frame: int
135         n_frames: int
136         period: float = 30.0
137         amplitude: float = 400.0
138
139
140     @dataclass(frozen=True)
141     class GapSpec:
142         """Dead-time between rallies ? what separates one window from the next.
143
144         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
145         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
146         end an active run so the next rally forms a distinct window.
147         """
148
149         start_frame: int
150         n_frames: int
151         kind: str = "still" # "still" | "occluded"
152
153
154     @dataclass(frozen=True)
155     class FixtureSpec:
156         """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.

```

```

157
158     ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
159     "labels.csv". For a synthetic fixture they line up with the rallies by construction,
160     which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
161     field-quality one (§6).
162     """
163
164     slug: str
165     description: str
166     fps: float
167     frame_width: float
168     frame_height: float
169     segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
170     gts: Tuple[Interval, ...]
171     net_midline: float = 640.0
172
173
174     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
175                       ) -> List[Tuple[int, int, float, float]]:
176         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
177         rows: List[Tuple[int, int, float, float]] = []
178         y_mid = frame_height / 2.0
179         for i in range(spec.n_frames):
180             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
181             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
182             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
183             rows.append((spec.start_frame + i, 1, x, y))
184         return rows
185
186
187     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
188                    ) -> List[Tuple[int, int, float, float]]:
189         """Dead-time rows: resting-visible (still) or absent (occluded)."""
190         rows: List[Tuple[int, int, float, float]] = []
191         y_mid = frame_height / 2.0
192         for i in range(spec.n_frames):
193             if spec.kind == "occluded":
194                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
195             elif spec.kind == "still":
196                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
197             else: # pragma: no cover - guarded by builder
198                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
199         return rows
200
201
202     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
203         """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
204         rows: List[Tuple[int, int, float, float]] = []
205         for seg in spec.segments:
206             if isinstance(seg, RallySpec):
207                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
208             elif isinstance(seg, GapSpec):
209                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
210             else: # pragma: no cover - guarded by dataclass shape
211                 raise TypeError(f"unknown segment type {type(seg).__name__}")
212         rows.sort(key=lambda r: r[0])
213         return rows
214
215
216     def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
217         """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp

```

```

floats."""
218     lines = ["Frame,Visibility,X,Y"]
219     for frame, vis, x, y in rows:
220         lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
221     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
222
223
224     def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
225         """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF. """
226         lines = ["start,end"]
227         for s, e in gts:
228             lines.append(f"{_fmt(s)},{_fmt(e)}")
229         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
230
231
232     # =====
233     # Built-in synthetic scenarios (the 3 committed slugs)
234     # =====
235
236
237     def _fr(seconds: float, fps: float) -> int:
238         return int(round(seconds * fps))
239
240
241     def builtin_specs() -> List[FixtureSpec]:
242         """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
243         windowing behaviour (one window / dead-time split / occlusion split)."""
244         fps = 30.0
245         fw, fh = 1280.0, 720.0
246         net = 640.0
247
248         # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
249         single = FixtureSpec(
250             slug="clean_single_rally",
251             description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
252             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
253             segments=(RallySpec(start_frame=0, n_frames=120),),
254             # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
255             gts=((0.0, 4.0),),
256         )
257
258         # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
259         # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
260         r0 = RallySpec(start_frame=0, n_frames=120)
261         gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
262         r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
263         multi = FixtureSpec(
264             slug="multi_rally_with_deadtime",
265             description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
266             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
267             segments=(r0, gap_still, r1),
268             gts=((0.0, 4.0), (7.0, 11.0)),
269         )
270
271         # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
272         # visibility break ? proves occlusion (not just low velocity) separates windows.
273         o0 = RallySpec(start_frame=0, n_frames=120)
274         gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")

```

```

275     o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
276     occ = FixtureSpec(
277         slug="occlusion_break",
278         description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
279         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
280         segments=(o0, gap_occ, o1),
281         gts=((0.0, 4.0), (7.0, 11.0)),
282     )
283
284     return [single, multi, occ]
285
286
287 def _spec_by_slug() -> Dict[str, FixtureSpec]:
288     return {s.slug: s for s in builtin_specs()}
289
290
291 # =====
292 # The shared replay (CONTROL / pure-stdlib path)
293 # =====
294
295
296 def _fmt(x: float) -> str:
297     """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
298     r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
299     if r == 0.0:
300         r = 0.0
301     return f"{r:.6f}"
302
303
304 def _round(x: float) -> float:
305     r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
306     return 0.0 if r == 0.0 else r
307
308
309 def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
310     """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
when present
311     (the pinned params travel with the fixture), else the module-level pinned defaults.
NEVER
312     ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
read here ?
313     ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
path; the
314     pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
PINNED_MAX_WINDOW)."""
315     if prov and "windowing" in prov:
316         w = prov["windowing"]
317         return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
318     return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
319
320
321 def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
322     """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
323
324     Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
325     ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
326
327     parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
328     cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
329

```

```

330     Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
331     control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
332     merges overlaps; NO numpy is touched.
333     """
334     fdir = _resolve_dir(slug_or_dir)
335     slug = fdir.name
336     traj_csv = fdir / "trajectory.csv"
337     labels_csv = fdir / "labels.csv"
338     prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
339
340     fps, fw = _fps_fw(prov, slug)
341     iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
342     proposal = _pinned_proposal(prov)
343
344     intervals, feature_rows = distill_local.build_candidates(
345         str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
346
347     # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
348     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
349     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
350     features: Dict[str, List[float]] = {
351         name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
352     }
353     gts = load_gt_intervals(str(labels_csv), strict=True)
354     vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
355
356     preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
357     sc = score(preds, gts, iou)
358
359     candidates: List[Dict[str, Any]] = []
360     for j, (s, e) in enumerate(intervals):
361         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
362         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
363         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
364         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
365
366     return {
367         "slug": slug,
368         "schema_version": CURRENT_SCHEMA,
369         "iou": _round(iou),
370         "candidates": candidates,
371         "control_segments": [[_round(s), _round(e)] for s, e in preds],
372         "score": {
373             "iou_threshold": _round(sc.iou_threshold),
374             "num_pred": int(sc.num_pred),
375             "num_gt": int(sc.num_gt),
376             "true_positives": int(sc.true_positives),
377             "false_positives": int(sc.false_positives),
378             "false_negatives": int(sc.false_negatives),
379             "precision": _round(sc.precision),
380             "recall": _round(sc.recall),
381             "f1": _round(sc.f1),
382             "mean_iou": _round(sc.mean_iou),
383             "mean_start_error": _round(sc.mean_start_error),
384             "mean_end_error": _round(sc.mean_end_error),
385         },

```

```

386         "gt_hash": gt_hash(gts),
387     }
388
389
390     # =====
391     # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
392     # =====
393
394
395     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
396                     ) -> Dict[str, Any]:
397         """The committed provenance block ? pins the windowing + privacy/licensing tags
398         ($4c, $5)."""
399         desc = ""
400         if isinstance(spec_or_dir, FixtureSpec):
401             desc = spec_or_dir.description
402         return {
403             "slug": slug,
404             "schema_version": CURRENT_SCHEMA,
405             "kind": "synthetic",
406             "license": "synthetic",
407             "minors_free": True,
408             "description": desc,
409             "fps": _round(fps),
410             "frame_width": _round(fw),
411             "iou": _round(iou),
412             "windowing": {
413                 "vt": PINNED_VT,
414                 "mg": PINNED_MERGE_GAP,
415                 "mwd": PINNED_MIN_WINDOW,
416                 "max_win": PINNED_MAX_WINDOW,
417             },
418             "paths": {
419                 "trajectory": "trajectory.csv",
420                 "labels": "labels.csv",
421                 "expected": "expected.json",
422             },
423             "note": (
424                 "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
425                 "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
426             )."
427         ),
428     }
429
430     def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
431                       iou: float = 0.5) -> Dict[str, Any]:
432         """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
433         COMMITTED
434         ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
435         construction).
436
437         Idempotent: rewrites the same bytes given the same committed inputs. Requires the
438         trajectory
439         + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
440         """
441         fdir = fixtures_dir / slug
442         traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
443         if not traj_csv.is_file() or not labels_csv.is_file():
444             raise FileNotFoundError(
445                 f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
446                 generate them")
447
448         # Provenance first (it pins the windowing the replay reads), then replay ? expected.
449         spec = _spec_by_slug().get(slug)

```

```

445         fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
446         prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
447         _write_json(fdir / "provenance.json", prov)
448
449         expected = replay_fixture(fdir)
450         _write_json(fdir / "expected.json", expected)
451         return expected
452
453
454     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
455         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
456         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
457         specs = builtin_specs()
458         fixtures_dir.mkdir(parents=True, exist_ok=True)
459         slugs: List[str] = []
460         for spec in specs:
461             fdir = fixtures_dir / spec.slug
462             fdir.mkdir(parents=True, exist_ok=True)
463             rows = make_synthetic_trajectory(spec)
464             write_trajectory_csv(rows, fdir / "trajectory.csv")
465             write_labels_csv(spec.gts, fdir / "labels.csv")
466             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
467             slugs.append(spec.slug)
468         _write_manifest(specs, fixtures_dir)
469         return slugs
470
471
472     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
473                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
474         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
475         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
476         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
477         if all_slugs:
478             targets = [d.name for d in sorted(fixtures_dir.iterdir())
479                       if d.is_dir() and (d / "trajectory.csv").is_file()]
480         elif slug:
481             targets = [slug]
482         else:
483             raise ValueError("refresh_snapshot needs --slug S or --all")
484         for s in targets:
485             freeze_fixture(s, fixtures_dir=fixtures_dir)
486         return targets
487
488
489     # =====
490     # Manifest entry derivation (shared ? used by both the synthetic and real-fixture
writers)
491     # =====
492
493
494     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
495         """A manifest row for a single fixture, derived from its provenance block."""
496         slug = prov["slug"]
497         return {
498             "slug": slug,
499             "fps": _round(float(prov["fps"])),
500             "frame_width": _round(float(prov["frame_width"])),
501             "windowing": dict(prov.get("windowing", {})),
502             "kind": prov.get("kind", "synthetic"),
503             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),

```

```

504         "paths": {
505             "dir": slug,
506             "trajectory": f"{slug}/trajectory.csv",
507             "labels": f"{slug}/labels.csv",
508             "expected": f"{slug}/expected.json",
509             "provenance": f"{slug}/provenance.json",
510         },
511     }
512
513
514     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
515         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
516         synthetic)."""
517         entries = load_manifest(fixtures_dir)
518         entries = [e for e in entries if e.get("slug") != prov["slug"]]
519         entries.append(_manifest_entry_for(prov))
520         entries.sort(key=lambda e: str(e.get("slug")))
521         _write_json(fixtures_dir / "manifest.json", entries)
522
523     # =====
524     # Manifest + loaders
525     # =====
526
527
528     def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
529         entries = [{
530             "slug": spec.slug,
531             "fps": _round(spec.fps),
532             "frame_width": _round(spec.frame_width),
533             "windowing": {
534                 "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
535                 "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
536             },
537             "kind": "synthetic",
538             "schema_version": CURRENT_SCHEMA,
539             "paths": {
540                 "dir": spec.slug,
541                 "trajectory": f"{spec.slug}/trajectory.csv",
542                 "labels": f"{spec.slug}/labels.csv",
543                 "expected": f"{spec.slug}/expected.json",
544                 "provenance": f"{spec.slug}/provenance.json",
545             },
546         } for spec in specs]
547         _write_json(fixtures_dir / "manifest.json", entries)
548
549
550     def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
551         """Load the fixtures manifest (returns [] if absent)."""
552         path = fixtures_dir / "manifest.json"
553         if not path.is_file():
554             return []
555         data = _read_json(path)
556         if not isinstance(data, list): # pragma: no cover - corrupt manifest
557             raise ValueError(f"{path}: manifest must be a JSON array")
558         return data
559
560
561     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
562         """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
563         a fixture
564         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
565         the
566         ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
567         forward-version

```

```

565 bump in one is never masked by an in-range value in the other (red-team #7)."""
566 fdir = fixtures_dir / slug
567 prov = _read_json(fdir / "provenance.json")
568 expected = _read_json(fdir / "expected.json")
569 for which, blob in (("expected", expected), ("provenance", prov)):
570     sv = int(blob.get("schema_version", 0))
571     if sv > CURRENT_SCHEMA:
572         raise ValueError(
573             f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
574             f"upgrade the harness")
575     return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
576
577
578 # =====
579 # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
580 # =====
581
582
583 def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
584     """Return a list of human-readable mismatch messages (empty == match).
585
586     Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
587     TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
588     equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
589     safe (the repo runs with core.autocrlf=true).
590
591     Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
592     per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
593     a fixed-field loop alone would ignore an added field (red-team #5)."""
594     out: List[str] = []
595
596     def exact(path: str, a: Any, b: Any) -> None:
597         if a != b:
598             out.append(f"{path}: {a!r} != {b!r} (exact)")
599
600     def approx(path: str, a: Any, b: Any) -> None:
601         try:
602             if abs(float(a) - float(b)) > _FLOAT_TOL:
603                 out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
604         except (TypeError, ValueError):
605             out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
606
607     # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
608     if set(recomputed) != set(committed):
609         out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
610
611     exact("slug", recomputed.get("slug"), committed.get("slug"))
612     exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
613     exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
614     approx("iou", recomputed.get("iou"), committed.get("iou"))
615
616     rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
617     if len(rc) != len(cc):
618         out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
619     else:

```

```

620         for i, (r, c) in enumerate(zip(rc, cc)):
621             approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
622             approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
623             rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
624             if set(rs) != set(cs):
625                 out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
626             else:
627                 for k in rs:
628                     if k == "net_crossings":
629                         exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
630                     else:
631                         approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
632
633         rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
634         if len(rseg) != len(cseg):
635             out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
636         else:
637             for i, (r, c) in enumerate(zip(rseg, cseg)):
638                 approx(f"control_segments[{i}][0]", r[0], c[0])
639                 approx(f"control_segments[{i}][1]", r[1], c[1])
640
641         rsc, csc = recomputed.get("score", {}), committed.get("score", {})
642         # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
643         if set(rsc) != set(csc):
644             out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
645         int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
646         float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
"mean_start_error", "mean_end_error")
647         for k in int_fields:
648             exact(f"score.{k}", rsc.get(k), csc.get(k))
649         for k in float_fields:
650             approx(f"score.{k}", rsc.get(k), csc.get(k))
651
652     return out
653
654
655
656 # =====
657 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
658 # =====
659
660
661 def _write_json(path: Path, obj: Any) -> None:
662     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
663     path.write_text(text + "\n", encoding="utf-8", newline="\n")
664
665
666 def _read_json(path: Path) -> Any:
667     with open(path, encoding="utf-8") as f:
668         return json.load(f)
669
670
671 def _resolve_dir(slug_or_dir: Any) -> Path:
672     if isinstance(slug_or_dir, Path):
673         return slug_or_dir
674     p = Path(str(slug_or_dir))
675     if p.is_dir():
676         return p
677     return FIXTURES_DIR / str(slug_or_dir)
678
679
680 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:

```

```

681     if prov and "fps" in prov and "frame_width" in prov:
682         return float(prov["fps"]), float(prov["frame_width"])
683     spec = _spec_by_slug().get(slug)
684     if spec is not None:
685         return spec.fps, spec.frame_width
686     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
687
688
689 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
690     prov_path = fdir / "provenance.json"
691     prov = _read_json(prov_path) if prov_path.is_file() else None
692     return _fps_fw(prov, slug)
693
694
695 # =====
696 # CLI
697 # =====
698
699
700 def _cmd_check(fixtures_dir: Path) -> int:
701     manifest = load_manifest(fixtures_dir)
702     if not manifest:
703         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
704         return 1
705     failures = 0
706     for entry in manifest:
707         slug = entry["slug"]
708         committed = _read_json(fixtures_dir / slug / "expected.json")
709         recomputed = replay_fixture(fixtures_dir / slug)
710         mismatches = compare_expected(recomputed, committed)
711         if mismatches:
712             failures += 1
713             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
f"(characterization = behaviour-locking, NOT correctness ? $6):")
714             for m in mismatches:
715                 print(f"    - {m}")
716         else:
717             sc = recomputed["score"]
718             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
719     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
720
721     f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
722     return 0 if failures == 0 else 1
723
724
725
726 def main(argv: Optional[Sequence[str]] = None) -> int:
727     ap = argparse.ArgumentParser(
728         description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
729     g = ap.add_mutually_exclusive_group(required=True)
730     g.add_argument("--freeze-synthetic", action="store_true",
731         help="generate synthetic trajectory+labels for all built-in
scenarios, then "
732             "freeze expected.json+provenance.json + write manifest.json")
733     g.add_argument("--refresh-snapshot", action="store_true",
734         help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
735     g.add_argument("--check", action="store_true",
736         help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
737     g.add_argument("--freeze-real", action="store_true",
738         help="(P1) de-attribute an OWNED real trajectory+labels into a

```

```

committable Tier-0 "
739             "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
740     ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
741             "--freeze-real (default: a random fx_<hex> id)")
742     ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
743     ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
744     # --freeze-real (P1 / M2 refusal gate) options:
745     ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
746     ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
747     ap.add_argument("--license", dest="license_", default=None,
748             help="--freeze-real) the source license tag (required, non-blank)")
749     ap.add_argument("--owned", action="store_true",
750             help="--freeze-real) assert owner-recorded Fork-A source
(required)")
751     ap.add_argument("--minors-free", dest="minors_free", action="store_true",
752             help="--freeze-real) assert the source contains NO minors
(required)")
753     ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
754     ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
755     ap.add_argument("--label", default="cleared_real",
756             help="--freeze-real) non-identifying scenario label ? STRICT slug
only "
757             "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
758     ap.add_argument("--source-note", dest="source_note", default="",
759             help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
760             "attribution back-channel, so it never reaches the committed
provenance "
761             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
762     ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
763             help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
764     ap.set_defaults(time_origin_reset=True)
765     ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
766             help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
767     args = ap.parse_args(list(argv) if argv is not None else None)
768
769     fixtures_dir: Path = args.fixtures_dir
770
771     if args.freeze_synthetic:
772         slugs = freeze_synthetic(fixtures_dir)
773         print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
'.join(slugs)}")
774         print(f"    -> {fixtures_dir}")
775         return 0
776
777     if args.refresh_snapshot:
778         if not args.slug and not args.all:
779             ap.error("--refresh-snapshot requires --slug S or --all")
780         # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
781         # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
782         if not (args.reason and args.reason.strip()):
783             ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-

```

```

stamp a real "
784             "regression; record the deliberate reason and review the visible
diff ($6).")
785         targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
786         print(f"[golden-fixtures] refreshed snapshot for: {' '.join(targets)}")
787         f" (reason: {args.reason})"
788         return 0
789
790     if args.freeze_real:
791         # The real-fixture subsystem (P1 + M2) lives in golden_real_fixtures (split
out); import it
792         # here so the CLI reaches the SAME freeze_real_fixture / RefusalError it always
has.
793         from backend.eval import golden_real_fixtures
794         missing = [n for n, v in (("--trajectory", args.trajectory),
795                                ("--labels", args.labels),
796                                ("--fps", args.fps),
797                                ("--frame-width", args.frame_width)) if v is None]
798         if missing:
799             ap.error("--freeze-real requires " + ", ".join(missing))
800         try:
801             expected = golden_real_fixtures.freeze_real_fixture(
802                 args.label, args.trajectory, args.labels,
803                 license=args.license_ if args.license_ is not None else "",
804                 minors_free=args.minors_free, owned=args.owned,
805                 fps=args.fps, frame_width=args.frame_width,
806                 fixtures_dir=fixtures_dir, slug=args.slug,
807                 time_origin_reset=args.time_origin_reset, source_note=args.source_note)
808         except golden_real_fixtures.RefusalError as e:
809             # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
zero.
810             print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
file=sys.stderr)
811             return 2
812             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
"
813                 f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
814             print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
to the "
815                 "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
(P2).")
816             return 0
817
818     # --check
819     return _cmd_check(fixtures_dir)
820
821
822     # =====
823     # Re-exports ? the real-fixture subsystem (P1 + M2) lives in ``golden_real_fixtures``
but
824     # is exposed here so every historical ``golden_fixtures.<name>`` import path stays
valid.
825     # =====
826
827     #: The names the real-fixture module owns; re-exported lazily (PEP 562 ``__getattr__``)
so the
828     #: ``golden_fixtures`` ? ``golden_real_fixtures`` cycle resolves regardless of import
order ?
829     #: ``golden_real_fixtures`` imports the shared primitives FROM this module, so an EAGER
bottom-of-
830     #: file re-export would fail when ``golden_real_fixtures`` is imported FIRST (its own
names would
831     #: not yet be defined when this module's import of them runs). The lazy hook defers the
import to

```

```

832     #: first attribute access, by which point BOTH modules are fully initialized.
833     _REAL_FIXTURE_EXPORTS: Tuple[str, ...] = (
834         "RefusalError", "freeze_real_fixture", "_scan_forbidden", "_new_surrogate_slug",
835         "_reset_trajectory_rows", "_reset_labels", "_SCHEMA_FILENAME_LITERALS",
836     )
837
838
839     def __getattr__(name: str) -> Any:
840         """Lazily resolve the re-exported real-fixture names from ``golden_real_fixtures``
841         (PEP 562)."""
842         if name in _REAL_FIXTURE_EXPORTS:
843             from backend.eval import golden_real_fixtures
844             return getattr(golden_real_fixtures, name)
845             raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
846
847     if __name__ == "__main__":
848         raise SystemExit(main())
849

```

10. assistant (2026-06-22T09:12:22.110Z)

Now let me search for evidence of actual golden video files or WASB caching:

11. user (2026-06-22T09:12:23.595Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures

12. user (2026-06-22T09:12:24.147Z)

```

1     """CLI: regression-test the production tool against golden-set ground truth.
2
3     Obtains ground truth from an annotation provider (a tier), scores the video's
4     stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5     detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7     Examples
8     -----
9     Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
11         --update-baseline
12
13     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
14         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
15     """
16     import os
17     import sys
18     import json
19     import argparse
20     import logging
21
22     from backend.infrastructure.database import Database
23     from backend.config import load_config
24     from backend.annotations import annotation_registry
25     from backend.eval import regression
26
27     def _default_db_path() -> str:
28         cfg = load_config()
29         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32     def build_parser() -> argparse.ArgumentParser:

```

```

33     p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34     p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35     p.add_argument("--tier", default="file",
36                     help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37     p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38     p.add_argument("--stage", choices=("candidates", "final"), default="final")
39     p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40     p.add_argument("--iou", type=float, default=0.5)
41     p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42     p.add_argument("--db", default=None)
43     p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44     p.add_argument("--update-baseline", action="store_true",
45                     help="Snapshot the current numbers as the new baseline instead of
checking.")
46     p.add_argument("--allow-missing-baseline", action="store_true",
47                     help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48     p.add_argument("--json", dest="json_out", default=None)
49     return p
50
51
52 def main(argv=None) -> int:
53     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54     args = build_parser().parse_args(argv)
55
56     db_path = args.db or _default_db_path()
57     if not os.path.exists(db_path):
58         print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59         return 2
60     db = Database(db_path)
61
62     result = regression.regress_with_provider(
63         db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64         stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65         tolerance=args.tolerance, baseline_path=args.baseline,
66     )
67
68     print(regression.format_result(result))
69
70     if args.update_baseline:
71         regression.save_baseline(result.video_id, result.stage, result.precision,
72                                 result.recall, result.f1, path=args.baseline,
73                                 iou_threshold=result.iou_threshold,
74                                 gt_fingerprint=result.gt_hash)
75         print(f"Baseline updated in {args.baseline}.")
76
77     if args.json_out:
78         with open(args.json_out, "w") as f:
79             json.dump(result.as_dict(), f, indent=2)
80             print(f"JSON written to {args.json_out}")
81
82     # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
83     if result.regressed:
84         return 1
85     if not result.has_baseline and not args.update_baseline and not
args.allow_missing_baseline:
86         print("ERROR: no baseline to compare against ? the gate cannot verify anything.

```

```

"
87         "Snapshot one with --update-baseline (then commit it), or pass "
88         "--allow-missing-baseline to accept this run.", file=sys.stderr)
89     return 3
90     return 0
91
92
93     if __name__ == "__main__":
94         raise SystemExit(main())
95

```

13. assistant (2026-06-22T09:12:26.721Z)

Let me check for actual fixture files and WASB caching:

14. user (2026-06-22T09:12:30.345Z)

```

total 32
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609 4221 Jun 22 14:36 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 clean_single_rally
-rw-r--r-- 1 avidu 197609 1557 Jun 22 14:36 manifest.json
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 multi_rally_with_deadtime
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 occlusion_break

```

15. user (2026-06-22T09:12:30.508Z)

(Bash completed with no output)

16. assistant (2026-06-22T09:12:37.196Z)

Now let me check the fixtures to understand their nature:

17. user (2026-06-22T09:12:38.354Z)

```

total 21K
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609  747 Jun 22 14:36 expected.json
-rw-r--r-- 1 avidu 197609   28 Jun 22 14:36 labels.csv
-rw-r--r-- 1 avidu 197609  648 Jun 22 14:36 provenance.json
-rw-r--r-- 1 avidu 197609 3.3K Jun 22 14:36 trajectory.csv

```

18. user (2026-06-22T09:12:38.731Z)

```

1      [
2      {
3          "fps": 30.0,
4          "frame_width": 1280.0,
5          "kind": "synthetic",
6          "paths": {
7              "dir": "clean_single_rally",
8              "expected": "clean_single_rally/expected.json",
9              "labels": "clean_single_rally/labels.csv",
10             "provenance": "clean_single_rally/provenance.json",
11             "trajectory": "clean_single_rally/trajectory.csv"
12         },
13         "schema_version": 1,
14         "slug": "clean_single_rally",
15         "windowing": {
16             "max_win": 0.0,
17             "mg": 2.0,
18             "mwd": 2.0,

```

```

19         "vt": 0.02
20     }
21 },
22 {
23     "fps": 30.0,
24     "frame_width": 1280.0,
25     "kind": "synthetic",
26     "paths": {
27         "dir": "multi_rally_with_deadtime",
28         "expected": "multi_rally_with_deadtime/expected.json",
29         "labels": "multi_rally_with_deadtime/labels.csv",
30         "provenance": "multi_rally_with_deadtime/provenance.json",
31         "trajectory": "multi_rally_with_deadtime/trajectory.csv"
32     },
33     "schema_version": 1,
34     "slug": "multi_rally_with_deadtime",
35     "windowing": {
36         "max_win": 0.0,
37         "mg": 2.0,
38         "mwd": 2.0,
39         "vt": 0.02
40     }
41 },
42 {
43     "fps": 30.0,
44     "frame_width": 1280.0,
45     "kind": "synthetic",
46     "paths": {
47         "dir": "occlusion_break",
48         "expected": "occlusion_break/expected.json",
49         "labels": "occlusion_break/labels.csv",
50         "provenance": "occlusion_break/provenance.json",
51         "trajectory": "occlusion_break/trajectory.csv"
52     },
53     "schema_version": 1,
54     "slug": "occlusion_break",
55     "windowing": {
56         "max_win": 0.0,
57         "mg": 2.0,
58         "mwd": 2.0,
59         "vt": 0.02
60     }
61 }
62 ]
63

```

19. assistant (2026-06-22T09:12:42.072Z)

Now let me search for WASB or cached trajectory/database files more broadly:

20. user (2026-06-22T09:12:43.885Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_cleanup_caches.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tests/test_wasb_cache.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/tools/cleanup_caches.py

```

21. user (2026-06-22T09:12:45.821Z)

```

Found 20 files limit: 20
.claude\worktrees\nightly-e2e\training\gen0\harness.py
.claude\worktrees\nightly-e2e\tools\generate_highlights.py

```

```
.claude\worktrees\nightly-e2e\tools\cleanup_caches.py
.claude\worktrees\nightly-e2e\tests\test_wsl_tilde_quoting.py
.claude\worktrees\nightly-e2e\tests\test_workspace.py
.claude\worktrees\nightly-e2e\tests\test_worker.py
.claude\worktrees\nightly-e2e\tests\test_windowing.py
.claude\worktrees\nightly-e2e\tests\test_wasb_runner.py
.claude\worktrees\nightly-e2e\tests\test_wasb_cache.py
.claude\worktrees\nightly-e2e\tests\test_upload_download.py
.claude\worktrees\nightly-e2e\tests\test_trajectory_hybrid_equivalence.py
.claude\worktrees\nightly-e2e\tests\test_training_gen0.py
.claude\worktrees\nightly-e2e\tests\test_tracknet_runner.py
.claude\worktrees\nightly-e2e\tests\test_tracknet_hybrid.py
.claude\worktrees\nightly-e2e\tests\test_stub_runner.py
.claude\worktrees\nightly-e2e\tests\test_storage.py
.claude\worktrees\nightly-e2e\tests\test_startup_checks.py
.claude\worktrees\nightly-e2e\tests\test_served_gate_a_regression.py
.claude\worktrees\nightly-e2e\tests\test_serve_contrast.py
.claude\worktrees\nightly-e2e\tests\test_reporting_harvest.py
```

22. assistant (2026-06-22T09:12:48.233Z)

Let me check the WASB cache test and the nightly regression script:

23. user (2026-06-22T09:12:49.468Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/bundle_ui.sh
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/doctor.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/install.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.ps1
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/nightly_regression.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/README.md
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/register_nightly_task.ps1
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/run_phase3_eval.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/run_process_and_stitch.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/scripts/uninstall.py
```

24. user (2026-06-22T09:12:49.984Z)

```
1      """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3      Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4      detection accumulation on replay, crash-truncation recovery, manifest
5      compatibility/invalidation, and atomic trajectory writes.
6
7      Also covers the streaming fast-path's pure helpers (synthetic frame addressing,
8      window builder, the sliding-buffer frame store, and the read_image shim) ? these
9      let the detector read frames straight from the decoded video instead of from ~46k
10     PNGs on disk, while preserving output exactly (byte-identical, verified).
11     """
12     import json
13     import os
14     import os.path as osp
15
16     import numpy as np
```

```

17 import pytest
18
19 from backend.pipeline.detectors import wasb_infer as wi
20
21
22 def _win(idx, frames):
23     """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
24     return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
25
26
27 def _write_windows(cache_dir, windows):
28     det_jsonl, _ = wi._cache_paths(cache_dir)
29     with open(det_jsonl, "a") as f:
30         wi._append_windows(f, windows)
31
32
33 # ----- #
34 # detection (de)serialization
35 # ----- #
36 def test_det_plain_and_back_roundtrip():
37     det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
38     plain = wi._det_plain(det)
39     assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
40
41     back = wi._dets_to_tracker([plain])
42     assert len(back) == 1
43     assert isinstance(back[0]["xy"], np.ndarray) # tracker does vector math on xy
44     assert back[0]["xy"].tolist() == [439.0, 64.5]
45     assert back[0]["scale"] == 0
46
47
48 # ----- #
49 # cache replay + resume index
50 # ----- #
51 def test_replay_accumulates_detections_across_windows(tmp_path):
52     cache = str(tmp_path / "c")
53     os.makedirs(cache)
54     # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
55     # frame 2 is empty in window 0 but detected in window 1.
56     _write_windows(cache, [
57         _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
58         _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
59     ])
60     windows_done, det = wi.load_and_clean_cache(cache)
61     assert windows_done == 2
62     assert det[0] == [[10, 20, 0.9, 0]]
63     assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]] # window-0 then window-1
64     assert det[2] == [[13, 23, 0.6, 0]]
65     assert det[3] == [] # seen but never detected
66
67
68 def test_resume_index_equals_window_count(tmp_path):
69     cache = str(tmp_path / "c")
70     os.makedirs(cache)
71     _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
72     windows_done, _ = wi.load_and_clean_cache(cache)
73     assert windows_done == 5
74
75
76 def test_empty_cache(tmp_path):
77     cache = str(tmp_path / "c")
78     os.makedirs(cache)
79     windows_done, det = wi.load_and_clean_cache(cache)
80     assert windows_done == 0 and dict(det) == {}
81

```

```

82
83 # ----- #
84 # crash robustness
85 # ----- #
86 def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
87     cache = str(tmp_path / "c")
88     os.makedirs(cache)
89     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
90                             _win(1, [(1, [[2, 2, 0.8, 0]])]))
91     det_jsonl, _ = wi._cache_paths(cache)
92     with open(det_jsonl, "a") as f: # simulate a crash mid-flush
93         f.write('{ "w":2, "f":[[2,[[3,3,']
94     windows_done, det = wi.load_and_clean_cache(cache)
95     assert windows_done == 2 # only the two clean windows
survive
96     assert det[1] == [[2, 2, 0.8, 0]]
97     # file is rewritten to the clean prefix -> appending window 2 stays contiguous
98     with open(det_jsonl) as f:
99         lines = [json.loads(ln) for ln in f if ln.strip()]
100     assert [o["w"] for o in lines] == [0, 1]

```

25. assistant (2026-06-22T09:12:52.086Z)

Now let me check the nightly_regression.py script:

26. user (2026-06-22T09:12:53.528Z)

```

1     """Nightly rally-quality regression tripwire.
2
3     Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4     seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5     the headline metrics against a FROZEN baseline
6     (`eval_baselines/nightly_baseline.json`).
7     Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
8     over-segmentation-guard regression.
9
10    This is the owner's dual-eval-set discipline made automatic (`memory/eval-dual-set-
regression`
11    + `docs/R2_EVAL_METRIC_DESIGN.md` $4.5): the growing GOLDEN-finalized set is re-run
every
12    night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
13    does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
14    no WASB re-extraction (those are cached and unchanged night to night).
15
16    Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
17
18    * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
19    to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
20    SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
21    * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
22    `docs/QUALITY_ITERATIONS.md` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
23    the nightly still names the milestone knobs and stays meaningful if production later
diverges.
24    * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
25    permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched

```

```

25     win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32     ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     -----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38         python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42         python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48     over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52     but no regression was found on the comparable intersection.
53     """
54     from __future__ import annotations
55
56     import argparse
57     import datetime as _dt
58     import json
59     import os
60     import sys
61     from dataclasses import dataclass, field, replace
62     from typing import Any, Dict, List, Optional, Tuple
63
64     # ----- #
65     # Paths (resolved relative to the repo root, so the script is cwd-robust).
66     # ----- #
67     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68     if REPO_ROOT not in sys.path:
69         sys.path.insert(0, REPO_ROOT) # importable as `python
scripts/nightly_regression.py` from anywhere
70     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74     DEFAULT_IOU = 0.5
75
76     # ----- #
77     # What we gate on (a tripwire ? focused, not every metric).
78     # * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
79     # * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.

```

```

80     # * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
81     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83     # ----- #
84     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89     AGG_SNAPSHOT: Tuple[str, ...] = (
90         "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91         "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
"dend_abs_median",
92     )
93     #: Per-video metrics snapshotted into the baseline.
94     PV_SNAPSHOT: Tuple[str, ...] = (
95         "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
"num_pred", "num_gt",
96     )
97
98     DEFAULT_METRIC_TOL = 0.005 # F1-like drop that counts as a regression (absorbs float
jitter)
99     DEFAULT_RATE_TOL = 0.010 # merge_rate rise that counts as a regression
100     DEFAULT_OVER_SEG_TOL = 0 # per-video split/merge increase allowed (strict)
101
102
103     # ----- #
104     # The two tracked configs.
105     # ----- #
106     @dataclass(frozen=True)
107     class NightlyConfig:
108         """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109
110         name: str
111         preset: Any # backend.eval.windowing.WindowingPreset
112         min_crossings: int = 0
113         note: str = ""
114
115
116     def tracked_configs() -> List[NightlyConfig]:
117         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).
118
119         All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
120         min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
121
122         * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
123         point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
124         DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
125         knob.
126         * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
127         the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
128         what we expect?" stays answerable even if a future change moves DEFAULT.
129         * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
130         the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the

```

```

131         dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
132         launch-time value). It is the reference DEFAULT used to be before the flip.""
133         from backend.eval.windowing import SERVED
134
135         default = NightlyConfig(
136             name="default", preset=SERVED, min_crossings=0,
137             note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
138         )
139         milestone = NightlyConfig(
140             name="milestone",
141             preset=replace(SERVED, name="milestone", max_window_duration=6.0,
142                 inactivity_end=1.5, inactivity_rally_thresh=0.20,
143                 inactivity_min_density=0.30),
144             min_crossings=0,
145             note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
146         )
147         baseline_off = NightlyConfig(
148             name="baseline_off",
149             preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
hard_cap=False,
150                 inactivity_end=0.0, inactivity_rally_thresh=0.08,
inactivity_min_density=0.34,
151                 blob_fallback=False, blob_factor=4.0),
152             min_crossings=0,
153             note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
154         )
155         return [default, milestone, baseline_off]
156
157
158     # ----- #
159     # Tolerances bundle.
160     # ----- #
161     @dataclass(frozen=True)
162     class Tolerances:
163         metric_tol: float = DEFAULT_METRIC_TOL
164         rate_tol: float = DEFAULT_RATE_TOL
165         over_seg_tol: int = DEFAULT_OVER_SEG_TOL
166
167         def to_dict(self) -> Dict[str, Any]:
168             return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,
169                 "over_seg_tol": self.over_seg_tol}
170
171
172     # ----- #
173     # Pure summary + comparison core (no I/O ? unit-testable).
174     # ----- #
175     def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
176         """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
177
178         Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
179         which is seeded but irrelevant to the gate) and per-video RAW metric values."""
180         agg = eval_result.get("aggregate", {}) or {}
181         aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
182         per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
183         return {
184             "n": int(agg.get("n", len(eval_result.get("rows", [])))),
185             "preset": eval_result.get("preset"),
186             "min_crossings": eval_result.get("min_crossings", 0),

```

```

187         "iou": eval_result.get("iou", DEFAULT_IOU),
188         "aggregate": aggregate,
189         "per_video": per_video,
190     }
191
192
193 @dataclass
194 class Finding:
195     """One comparison outcome line (regression / improvement / stale note)."""
196
197     config: str
198     scope: str          # "aggregate" | "per_video" | "config"
199     metric: str
200     kind: str           # "regression" | "improvement" | "stale"
201     baseline: Optional[float] = None
202     current: Optional[float] = None
203     video: Optional[str] = None
204
205     @property
206     def delta(self) -> Optional[float]:
207         if self.baseline is None or self.current is None:
208             return None
209         return self.current - self.baseline
210
211     def message(self) -> str:
212         loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
f"{self.config}/{self.metric}"
213         if self.kind == "stale":
214             return f"[STALE] {self.config}: {self.metric}"
215         d = self.delta
216         ds = f"{d:+.4f}" if d is not None else "?"
217         b = "?" if self.baseline is None else f"{self.baseline:.4f}"
218         c = "?" if self.current is None else f"{self.current:.4f}"
219         tag = "REGRESSION" if self.kind == "regression" else "improvement"
220         return f"[{tag}] {loc}: {b} ? {c} ({ds})"
221
222
223 @dataclass
224 class ConfigComparison:
225     config: str
226     status: str = "ok"          # "ok" | "regression" | "stale"
227     findings: List[Finding] = field(default_factory=list)
228     added_videos: List[str] = field(default_factory=list)
229     removed_videos: List[str] = field(default_factory=list)
230     aggregate_gated: bool = True # False when preset/corpus changed ? aggregate not
comparable
231
232     @property
233     def regressions(self) -> List[Finding]:
234         return [f for f in self.findings if f.kind == "regression"]
235
236     @property
237     def improvements(self) -> List[Finding]:
238         return [f for f in self.findings if f.kind == "improvement"]
239
240
241     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
242         """Are the two stored preset dicts the same windowing definition? (name is
cosmetic)."""
243         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
244             return base_preset == cur_preset
245         keys = set(base_preset) | set(cur_preset)
246         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
247
248

```

```

249     def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
250                       tols: Tolerances) -> ConfigComparison:
251         """Compare one config's current summary to its baseline summary.
252
253         * If the preset definition or the net-crossings gate changed, the whole config is
254         STALE
255         (numbers aren't comparable) ? flag it, skip gating.
256         * If the corpus (video set) changed, the aggregate is not comparable (different
257         videos) ?
258         skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
259         the
260         INTERSECTION (a code regression on a shared video must still trip the wire).
261         """
262         cmp = ConfigComparison(config=name)
263
264         # 1) Config-definition drift ? stale, not gateable.
265         if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
266             or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
267             cmp.status = "stale"
268             cmp.aggregate_gated = False
269             cmp.findings.append(Finding(config=name, scope="config", metric="preset",
270 kind="stale"))
271         return cmp
272
273         base_pv = base_sum.get("per_video", {})
274         cur_pv = cur_sum.get("per_video", {})
275         base_names, cur_names = set(base_pv), set(cur_pv)
276         cmp.added_videos = sorted(cur_names - base_names)
277         cmp.removed_videos = sorted(base_names - cur_names)
278         corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
279
280         # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
281         comparable).
282         if corpus_changed:
283             cmp.aggregate_gated = False
284             cmp.status = "stale"
285             cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
286 kind="stale"))
287         else:
288             base_agg = base_sum.get("aggregate", {})
289             cur_agg = cur_sum.get("aggregate", {})
290             for m in GATED_HIGHER:
291                 b, c = base_agg.get(m), cur_agg.get(m)
292                 if b is None or c is None:
293                     continue
294                 if c < b - tols.metric_tol:
295                     cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
296                 elif c > b + tols.metric_tol:
297                     cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
298             for m in GATED_LOWER:
299                 b, c = base_agg.get(m), cur_agg.get(m)
300                 if b is None or c is None:
301                     continue
302                 if c > b + tols.rate_tol:
303                     cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
304                 elif c < b - tols.rate_tol:
305                     cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
306
307         # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
308         regressions).
309         for vid in sorted(base_names & cur_names):
310             for m in PER_VIDEO_GUARD:
311                 b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
312                 if b is None or c is None:
313                     continue

```

```

307         if c > b + tols.over_seg_tol:
308             cmp.findings.append(Finding(name, "per_video", m, "regression",
float(b), float(c), video=vid))
309
310         if cmp.regressions:
311             cmp.status = "regression"
312         elif cmp.status != "stale":
313             cmp.status = "ok"
314         return cmp
315
316
317     @dataclass
318     class NightlyResult:
319         comparisons: List[ConfigComparison] = field(default_factory=list)
320         skipped: Dict[str, List[str]] = field(default_factory=dict) # config -> manifest
videos not scored
321
322         @property
323         def has_regression(self) -> bool:
324             return any(c.status == "regression" for c in self.comparisons)
325
326         @property
327         def has_stale(self) -> bool:
328             return any(c.status == "stale" for c in self.comparisons)
329
330         def overall_status(self) -> str:
331             if self.has_regression:
332                 return "REGRESSION"
333             if self.has_stale:
334                 return "STALE"
335             return "PASS"
336
337         def exit_code(self) -> int:
338             if self.has_regression:
339                 return 1
340             if self.has_stale:
341                 return 4
342             return 0
343
344
345         def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
346                       tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
347             """Compare every tracked config's current summary to the baseline summary."""
348             res = NightlyResult(skipped=skipped or {})
349             base_cfgs = baseline.get("configs", {})
350             cur_cfgs = current.get("configs", {})
351             for name in cur_cfgs:
352                 if name not in base_cfgs:
353                     cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
354                     cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
355                     res.comparisons.append(cmp)
356                     continue
357             res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
358         return res
359
360
361     # ----- #
362     # Report formatting (pure).
363     # ----- #
364     def _fmt(v: Optional[float]) -> str:
365         return "?" if v is None else f"{v:.4f}"
366

```

```

367
368     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
369                       result: NightlyResult, tols: Tolerances, date: str) -> str:
370         L: List[str] = []
371         status = result.overall_status()
372         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
373         L.append(f"# Nightly rally-quality regression ? {date}")
374         L.append("")
375         L.append(f"***Status: {badge}** . exit code {result.exit_code()}")
376         L.append("")
377         L.append(f"- HEAD: `{current.get('git_commit', '?')}` . baseline: "
378                 f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')})")
379         L.append(f"- Corpus manifest: `{current.get('manifest', '?')}` .
IoU={current.get('iou', DEFAULT_IoU)}")
380         L.append(f"- Tolerances: metric  $\pm$ {tols.metric_tol}, rate  $\pm$ {tols.rate_tol}, "
381                 f"per-video over-seg  $\pm$ {tols.over_seg_tol}")
382         L.append("")
383
384         # Up-front regression / stale summary.
385         regs = [f for c in result.comparisons for f in c.regressions]
386         if regs:
387             L.append("## ? Regressions")
388             L.append("")
389             for f in regs:
390                 L.append(f"- {f.message()}")
391             L.append("")
392         if result.has_stale:
393             L.append("## ? Stale / not comparable")
394             L.append("")
395             for c in result.comparisons:
396                 if c.status == "stale":
397                     if c.added_videos:
398                         L.append(f"- `{c.config}`: corpus added {c.added_videos}")
399                     if c.removed_videos:
400                         L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
401                     if not c.added_videos and not c.removed_videos:
402                         L.append(f"- `{c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
403                     L.append(f"- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
404                     L.append("")
405
406         # Skipped (no silent caps).
407         skipped_any = {k: v for k, v in result.skipped.items() if v}
408         if skipped_any:
409             L.append("## ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
410             L.append("")
411             for cfg, vids in skipped_any.items():
412                 L.append(f"- `{cfg}`: {vids}")
413             L.append("")
414
415         # Per-config detail.
416         cur_cfgs = current.get("configs", {})
417         base_cfgs = baseline.get("configs", {})
418         for cmp in result.comparisons:
419             cur = cur_cfgs.get(cmp.config, {})
420             base = base_cfgs.get(cmp.config, {})
421             L.append(f"## Config `{cmp.config}` ? {cur.get('note', '')}")
422             L.append("")
423             L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})")
424             L.append("")

```

```

425         cagg = cur.get("aggregate", {})
426         bagg = base.get("aggregate", {})
427         L.append("| metric | baseline | current | ? | gated |")
428         L.append("|---|---|---|---|---|")
429         for m in AGG_SNAPSHOT:
430             b, c = bagg.get(m), cagg.get(m)
431             d = (c - b) if (b is not None and c is not None) else None
432             gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
433             L.append(f"| {m} | {_fmt(b)} | {_fmt(c)} | {_fmt(d)} | {gated} |")
434             L.append("")
435             # Per-video over-seg guard table.
436             cpv = cur.get("per_video", {})
437             bpv = base.get("per_video", {})
438             names = sorted(set(cpv) | set(bpv))
439             if names:
440                 L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
441                 L.append("|---|---|---|---|")
442                 for v in names:
443                     cb, cc = bpv.get(v, {}), cpv.get(v, {})
444                     tf = f"{_fmt(cb.get('tol_f1'))}{_fmt(cc.get('tol_f1'))}"
445                     sp = f"{cb.get('split_count', '?')}{cc.get('split_count', '?')}"
446                     mg = f"{cb.get('merge_count', '?')}{cc.get('merge_count', '?')}"
447                     flag = " ?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""
448                     L.append(f"| {v}{flag} | {tf} | {sp} | {mg} |")
449                     L.append("")
450             imps = cmp.improvements
451             if imps:
452                 L.append("_Improvements over baseline (consider `--update-baseline` to
ratchet the floor up):_")
453                 for f in imps:
454                     L.append(f"- {f.message()}")
455                 L.append("")
456
457             L.append("----")
458             L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change ")
459             "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
460             return "\n".join(L)
461
462
463     # ----- #
464     # I/O shell.
465     # ----- #
466     def _git_commit() -> str:
467         import subprocess
468         try:
469             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
capture_output=True, text=True, timeout=10)
470             return out.stdout.strip() or "?"
471         except Exception:
472             return "?"
473
474
475
476     def _display_path(path: str) -> str:
477         """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
478         in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
479         Metadata only ? the comparison never reads it."""
480         norm = path.replace("\\", "/")
481         parent = os.path.basename(os.path.dirname(norm))
482         if parent == "output":

```

```

483         return "output/" + os.path.basename(norm)
484     try:
485         rel = os.path.relpath(path, REPO_ROOT)
486         if not rel.startswith(".."):
487             return rel.replace("\\", "/")
488     except ValueError:
489         pass
490     return norm
491
492
493     def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
494                   iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
495         """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
496 per-config).
497         ``snapshot`` is the serialisable structure stored as / compared to the baseline.
498         ``skipped`` lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
499 surfaced in the report so the corpus can't silently shrink."""
500         from backend.eval import rally_seg_eval
501
502         golden = rally_seg_eval.load_golden(manifest, features_dir)
503         manifest_names = [v["name"] for v in golden]
504         if not golden:
505             raise RuntimeError(f"no golden videos loaded from {manifest}")
506
507         snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou,
508 "configs": {}}
509         skipped: Dict[str, List[str]] = {}
510         scored_any = False
511         for cfg in configs:
512             result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou,
513 min_crossings=cfg.min_crossings)
514             summary = summarize_run(result)
515             summary["note"] = cfg.note
516             snapshot["configs"][cfg.name] = summary
517             scored = set(summary["per_video"])
518             skipped[cfg.name] = [n for n in manifest_names if n not in scored]
519             scored_any = scored_any or bool(scored)
520         if not scored_any:
521             raise RuntimeError(
522 f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
523 present "
524 f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?")
525         return snapshot, skipped
526
527
528     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
529         if not os.path.exists(path):
530             return None
531         with open(path, encoding="utf-8") as fh:
532             return json.load(fh)
533
534
535     def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
536         out = dict(snapshot)
537         out["generated_at"] = _dt.date.today().isoformat()
538         out["git_commit"] = _git_commit()
539         out["tolerances"] = tols.to_dict()
540         out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
541 "python scripts/nightly_regression.py --update-baseline` after an
542 intended "
543 "improvement or a corpus change. Tied to the LOCAL golden corpus (not
544 committed).")

```

```

540     os.makedirs(os.path.dirname(path), exist_ok=True)
541     tmp = path + ".tmp"
542     with open(tmp, "w", encoding="utf-8") as fh:
543         json.dump(out, fh, indent=2, sort_keys=True)
544         fh.write("\n")
545     os.replace(tmp, path)
546
547
548     def build_parser() -> argparse.ArgumentParser:
549         p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire
550 (offline, CPU).")
551         p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
552         p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
553             help="dir holding <name>_golden_features.json")
554         p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON
555 path")
556         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
557 are written")
558         p.add_argument("--iou", type=float, default=DEFAULT_IIOU)
559         p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
560             help="F1-like aggregate drop that counts as a regression")
561         p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
562             help="merge_rate rise that counts as a regression")
563         p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
564             help="per-video split/merge-count increase allowed before it's a
565 regression")
566         p.add_argument("--update-baseline", action="store_true",
567             help="snapshot the current numbers as the new baseline instead of
568 checking")
569         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
570 for tests)")
571         p.add_argument("--no-report", action="store_true", help="skip writing the dated
572 report file")
573         return p
574
575
576     def main(argv: Optional[List[str]] = None) -> int:
577         import logging
578         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
579         log = logging.getLogger("nightly_regression")
580         args = build_parser().parse_args(argv)
581         tols = Tolerances(metric_tol=args.metric_tol, rate_tol=args.rate_tol,
582 over_seg_tol=args.over_seg_tol)
583         date = args.date or _dt.date.today().isoformat()
584
585         if not os.path.exists(args.manifest):
586             log.error("manifest not found: %s ? the golden corpus must be present locally.",
587 args.manifest)
588             return 2
589
590         try:
591             snapshot, skipped = run_corpus(tracked_configs(), args.manifest,
592 args.features_dir, args.iou)
593         except RuntimeError as e:
594             log.error("%s", e)
595             return 2
596
597         snapshot["git_commit"] = _git_commit()
598         snapshot["generated_at"] = date
599
600         if args.update_baseline:
601             save_baseline(args.baseline, snapshot, tols)
602             log.info("baseline snapshot written to %s", args.baseline)
603             for cfg, s in snapshot["configs"].items():
604                 agg = s["aggregate"]
605                 log.info(" %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",

```

```

595             cfg, s["n"], agg.get("tol_f1") or 0.0, agg.get("f1@0.5") or 0.0,
596             agg.get("merge_rate") or 0.0)
597         return 0
598
599     baseline = load_baseline(args.baseline)
600     if baseline is None:
601         log.error("no baseline at %s ? snapshot one with --update-baseline (then commit
it).", args.baseline)
602         return 3
603
604     result = compare_all(baseline, snapshot, tols, skipped)
605     report = format_report(baseline, snapshot, result, tols, date)
606
607     if not args.no_report:
608         os.makedirs(args.report_dir, exist_ok=True)
609         md_path = os.path.join(args.report_dir, f"{date}.md")
610         json_path = os.path.join(args.report_dir, f"{date}.json")
611         with open(md_path, "w", encoding="utf-8") as fh:
612             fh.write(report + "\n")
613         with open(json_path, "w", encoding="utf-8") as fh:
614             json.dump({"date": date, "status": result.overall_status(),
615                       "exit_code": result.exit_code(), "current": snapshot,
616                       "regressions": [f.message() for c in result.comparisons for f in
c.regressions]}},
617                       fh, indent=2, sort_keys=True)
618         log.info("report written to %s", md_path)
619
620     print(report)
621     code = result.exit_code()
622     if code == 1:
623         log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                 sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625     elif code == 4:
626         log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627     else:
628         log.info("nightly regression check PASSED")
629     return code
630
631
632 if __name__ == "__main__":
633     raise SystemExit(main())
634

```

27. assistant (2026-06-22T09:12:55.680Z)

Now let me check if the manifest and golden features are local or if there's documentation about them:

28. user (2026-06-22T09:12:57.531Z)

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/manifest.json

29. user (2026-06-22T09:12:59.081Z)

```

.claude\worktrees\nightly-e2e\scripts\nightly_regression.py:13:no WASB re-extraction (those are
cached and unchanged night to night).
.claude\worktrees\nightly-e2e\scripts\nightly_regression.py:28:outside the repo +
`output/<name>_golden_features.json`); the corpus is NOT committed. So the
.claude\worktrees\nightly-e2e\scripts\nightly_regression.py:72:DEFAULT_MANIFEST =
os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
.claude\worktrees\nightly-e2e\scripts\nightly_regression.py:521:           f"and do the
{features_dir}/<name>_golden_features.json files have 'gts'?"

```

```
.claude\worktrees\nightly-e2e\scripts\nightly_regression.py:552:  
holding <name>_golden_features.json")
```

help="dir

30. assistant (2026-06-22T09:13:01.243Z)

Let me check if the manifest and golden features exist locally:

31. user (2026-06-22T09:13:02.128Z)

```
ls: cannot access 'C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees\nightly-e2e/output/': No such file or directory
```

32. user (2026-06-22T09:13:03.101Z)

```
1      # Golden Regression Fixtures ? video-free, non-attributable regression suite
2
3      > **Status:** `IN PROGRESS` ? owner-gated . **Owner:** Avi . **Created:** 2026-06-16 .
**Last updated:** 2026-06-16 . **Legal:** owner **accepted the researched risk** 2026-06-16
(condition §3 stand; this is the owner's informed acceptance, **not** a retained-counsel
opinion)
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE, per archive policy)
5      > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
6      > **Relation to existing docs:** EXTENDS (does not supersede)
`docs/GOLDEN_DATA_SHARING.md` (#175); reuses the eval stack documented in `docs/EVALUATION.md` /
`docs/EXPERIMENT_HARNESS.md`; quality floor ties to `docs/QUALITY_ITERATIONS.md`.
7      > **Honesty note:** §3 (Legal) is **researched general information, NOT legal advice** ?
it gates on retained counsel (§8). §2/§4/§7 mark which claims are **[verified]** against code vs
**[design]** proposals.
8
9      ---
10
11     ## 0. TL;DR
12
13     Let a contributor on **another machine with no video, GPU, WSL, or DB** run the rally-
segmentation regression suite from **committed structured files**, so refactors and non-quality
PRs get a red/green side-effect signal. Two layers, built once and grown by accretion:
14
15     1. **Mechanics** ? freeze the deterministic pipeline at the **post-detector trajectory
seam** and replay it through the *already-shipped* eval stack (`experiment.py` / `metrics.py` /
`windowing.py` / `regression.py`), with two auto-discovered gates: a
**characterization/snapshot** gate (catches any behaviour change) and a **quality-floor** gate
(catches quality drops).
16     2. **Privacy/legal filter (SEALED-FIXTURES)** ? only **de-attributed, owner-source,
biometric-free** numbers are ever committed; richer/sensitive streams stay key-gated or out-of-
repo. Non-attributability comes from **deletion-at-source**, not encryption. ? "Minimized" means
**stream minimization** (shuttle-only, no person/audio/pose) ? **NOT** reduction of the shuttle
stream to ~5 scalars: the de-attributed per-frame `trajectory.csv` IS committed (see §4c + its
risk-acceptance re-confirmation note).
17
18     **The convergence that makes this clean:** the *privacy* answer (publish shuttle-only,
drop person/pose streams) and the *determinism* answer (freeze only the `CONTROL` path ? which
runs **no numpy execution**, keeping the numpy/BLAS learned scorer off the cross-machine gate)
point at the **same minimal public tier**. That coincidence is the spine of the design. (numpy
is still *imported* transitively via `experiment` and must be installed; the CONTROL replay just
never calls into it.)
19
20     **Two findings already actionable, independent of the rest:**
21     - ? **Live de-attribution bug** *(verified)*: real player/venue names are committed
today in `eval_baselines/heuristic_lovo_n6.json`,
`eval_baselines/gemini_wrapper_2026-06-10.json`, and `backend/eval/eval_partial_labels.py` (the
last also has a personal `C:/Users/avidu/...` path) ? and in git history. ? **Deliverable P0.**
22     - ? **Determinism trap** *(verified, corroborated by two independent red-teams)*: the
```

learned variants (`shuttle\_only`, `fusion`) train a numpy/OpenBLAS `DYNAMIC\_ARCH` logistic regression whose decisions flip near the 0.5 threshold across CPUs ? would false-RED on a contributor's machine. ? the cross-machine gate must be **CONTROL/static-only**.

23
24 ---
25
26 ## 1. Problem & goal
27
28 The rally-quality measurement is split across two machines by capability
(`docs/GOLDEN\_DATA\_SHARING.md`):
29 - `backend/eval/fusion\_golden.py` **extracts** `\_golden\_features.json` ? needs the
video + trajectory CSV + labels + GPU/WSL (GPU box only).
30 - `backend/eval/fusion\_compare.py` / `backend/eval/ablation.py` **re-score** those JSONs
on **pure CPU, no video** ? any machine.
31
32 The CPU re-scoring is **already** video-free, but the inputs live in gitignored `output`/
/private OneDrive, so the regression tests that depend on them **skip everywhere**
(`tests/test\_ablation.py::test\_litmus\_reproduces\_gate\_a`, `tests/test\_fusion\_compare.py`). The
committed `eval\_baselines/` baselines exist but `regression.py` still scores DB-stored
predictions (needs a pipeline run on the video).
33
34 **Goal:** commit a tiny, version-tagged, **numeric-only, non-attributable** per-fixture
corpus + auto-discovered gates so a clean `git clone` / CI gets a real red/green regression
signal with **zero video/GPU/DB**, while never leaking attributable or sensitive data.
35
36 ---
37
38 ## 2. Decisions locked
39
40 | # | Decision | Source | Implication |
41 |---|-----|-----|-----|
42 | D1 | **Corpus is owner-recorded / owned** | owner answer 2026-06-16 | Legal analysis
takes **Fork A** (clean): no third-party copyright/ToS infringement; concern is trade-
secret/moat + privacy. Strongest non-attributability (no public reference to correlate against).
|
43 | D2 | **Repo private now, may open-source later** | owner answer 2026-06-16 | Design
for **public-grade hygiene from day one** ? git history is permanent; nothing scrubable-later. |
44 | D3 | **Adults, consent not formalized** | owner answer 2026-06-16 | **Person/pose/gait**
streams stay OUT of the public tier (DPDP/GDPR derived-personal-data risk without consent).
Public tier = **shuttle + abstract intervals + non-biometric features only**. |
45 | D4 | **Freeze seam = post-detector trajectory** (design; SHIPPED differs ? see ?) |
workflow-1 synthesis | Widest deterministic surface; reuse the shipped eval stack. ?
Correction (audit 2026-06-16): the shipped freeze **commits** the de-attributed per-frame
`(Frame,Visibility,X,Y)` `trajectory.csv` (shuttle-only, time-origin reset) ? it is **NOT**
minimized to ~5 aggregate scalars. See §4c (and its risk-acceptance re-confirmation note). |
46 | D5 | **Cross-machine gate is CONTROL/static-only** (design) | both red-teams
[verified cause] | Learned variants (numpy/BLAS) are **owner-box/Tier-1 only**, asserted with
tolerance, never as a cross-machine byte-snapshot. |
47 | D6 | **Non-attributability = deletion-at-source, encryption = public-only barrier**
(design) | workflow-2 synthesis | A contributor who runs the tests necessarily reads
plaintext+key (DRM/analog-hole). So we **delete** attribution, not hide it. Encryption only
stops the no-key public. |
48 | D7 | **Two tiers** (design) | workflow-2 synthesis | **Tier-0** public/no-key
(shuttle-only, de-attributed). **Tier-1** key-gated/out-of-repo (person/audio, full corpus),
skips cleanly when absent. |
49
50 ---
51
52 ## 3. Legal foundation (researched ? gates on counsel, §8)
53
54 **Core verdict** (settled across IN/US/EU): the derived numbers ? per-frame shuttle
(x,y), rally start/end intervals, audio/court/aggregate-person feature values ? **do NOT** inherit
the source video's copyright. They are uncopyrightable **facts/measurements**, not authored
expression, and not a §101 "derivative work."
55

56 | Jurisdiction | Copyright of the data | Database right | Contract / ToS | Net |
57 |---|---|---|---|---|
58 | ****India**** (primary) | Not copyrightable as facts (**EBC v. D.B. Modak**); but
compilations get thin protection on a low "skill/labour" bar (**Burlington**, **OLX v. Padawan**) ?
only bites if ****lifted from a 3rd-party dataset****, not self-derived | ****None**** (no sui-generis
right) ? a real permissiveness edge | Anti-scraping ToS (untested in court) + ****IT Act**
s.43/s.66** civil/criminal exposure for 3rd-party ingestion | ****Owned-source + minors-excluded +**
no person/pose = clean** |
59 | ****US**** | Not copyrightable (**Feist**; §102(b); **NBA v. Motorola**) | None (**Feist**
killed sweat-of-the-brow) | ****Dominant risk.**** ToS enforceable where copyright/CFAA fail
(**ProCD**, **hiQ** ? injunction + ****forced deletion****). Remedy for redistribution is takedown, not
nominal damages | Publishable as facts ****iff**** owned/permissioned source + no barring ToS + no
person/biometric/minor stream |
60 | ****EU / UK**** | Not copyrightable (**Football Dataco v. Yahoo**) | ****Genuinely**
contestable.** **Football Dataco v. Sportradar**: recording pre-existing on-court facts =
****obtaining****, not "creating" ? our "we created it" premise is ****overconfident****; harm test =
CV-Online Latvia | *Ryanair v. PR Aviation*: ToS can ban extraction/redistribution even with no
IP right | Get EU+UK advice before redistributing numeric streams at scale |
61
62 ****Provenance is the decisive fork.**** ****Fork A** (owner-recorded, unpublished) ? the only
path safe to commit publicly; collapses to a trade-secret **choice** (Indian breach-of-confidence
/ Contract Act 1872 / IT Act s.72 ? ****not**** US DTSA) + privacy. ****Fork B**
(broadcast/YouTube/scraped) ? do ****not**** commit; the extraction act needs a license or
now-****contested**** US fair use (**Thomson Reuters v. Ross** 2025; USCO May-2025), and unlawful
acquisition independently sinks it (**Bartz v. Anthropic**, ~\$1.5B settlement). Caveat: owner
footage ****shot at a third party's event**** (tournament/club) carries
ticket/venue/organiser/broadcaster terms ? treat as Fork B until cleared.
63
64 ****Publish-by-stream (privacy):****
65 - ? ****Publish-OK:**** shuttle (x,y)+Visibility (inanimate object), abstract rally/GT
****intervals**** (identity-stripped), non-biometric audio/court/aggregate-person features detached
from identity.
66 - ?? ****Conditional (effectively never in v1):**** person bounding boxes that **persistently*
single out a player ? flips to GDPR Art.9 the moment it identifies. This architecture is prone
to that flip ? treat per-player tracking as a tripwire.
67 - ? ****Never publish:**** pose/skeleton/****gait**** (re-identifies at 80?95% even with faces
blurred ? **Weiss et al. 2025**), face crops, any named-player?movement mapping. Matches the
existing "gait/biometrics strictly future (GDPR Art.9)" guardrail.
68 - ? ****Minors flip everything:**** DPDP "child" = under 18; s.9(3) ****prohibits behavioural**
****monitoring**** (the Rule-12 exemptions don't cover this project). A persistent per-player
signature on a minor is, conservatively, prohibited. ****Exclude minors**** from any persisted
stream beyond inanimate shuttle/interval data.
69
70 ****Non-attributability is relative, not absolute.**** A per-video trajectory is a
fingerprint (de Montjoye: 4 points ? 95% re-ID). It's only non-attributable because the ****Fork-A**
source stays unpublished** (no public reference to correlate against). Per **EDPS v. SRB** (CJEU
C-413/23 P, 2025) it ****remains personal data to the holder**** who can re-run the detector. So:
conditional, revocable (publish the source and the link snaps back), forward-looking (re-assess
as the public-video environment grows). Never market it as "anonymous."
71
72 ***Key authorities:** Feist 499 U.S. 340; NBA v. Motorola 105 F.3d 841; 17 U.S.C.
§101/§102(b); Sega v. Accolade 977 F.2d 1510; Football Dataco v. Yahoo (C-604/10) & v.
Sportradar (C-173/11); BHB v. William Hill (C-203/02); CV-Online Latvia (C-762/19); Ryanair v.
PR Aviation (C-30/14); Thomson Reuters v. Ross (D. Del. 2025); EBC v. D.B. Modak; DPDP Act 2023
s.3/s.9 + Rules 2025; EDPS v. SRB (C-413/23 P). Full URLs in the research artifact (§10).
73
74 ---
75
76 **## 4. Architecture (design)**
77
78 **### 4a. Freeze seam + reuse map**
79 Freeze at the ****trajectory CSV ? CONTROL-path replay****. One shared helper
`backend/eval/golden\_fixtures.py::replay\_fixture()` is the **single** code path used by both the
freeze tool and both gates. As ****shipped****, that path is (per the module docstring):
80 `parse_trajectory_csv ? distill_local.build_candidates(proposal=(vt,mg,mwd)) ?

```

VideoCandidates(5 shuttle cols only) ? experiment.predict(CONTROL) [keep all ? merge_overlaps] ?
metrics.score`.
81     Reuses verbatim: `metrics.score/interval_iou`,
`experiment.predict/CONTROL/VideoCandidates`, `distill_local.build_candidates`,
`tracknet_runner.parse_trajectory_csv`/`trajectory_to_action_windows`, `rally_gate` net-
crossings (inside `build_candidates`), `gt_loader.load_gt_intervals` (strict),
`regression.gt_hash`, `eval_baselines`/`committed-baseline precedent`.
82     > ? Correction (audit 2026-06-16): the replay does NOT import `windowing.py` ?
there is no `windowing.build_windows(preset)` / `WindowingPreset` / "default SERVED" call. The
windowing params travel as a raw 3-tuple `(vt, mg, mwd)` pinned in each fixture's
`provenance.json` and passed straight to `build_candidates`; the harness deliberately avoids the
named `windowing.SERVED` constant so a future SERVED retune can't silently churn fixtures.
(`segmentation_metrics`, `experiment.compare/Variant.spec_hash`, `regression.save_baseline`,
`eval_stats` are not on the M1 replay path either ? the quality-floor gate M4 will add the
baseline/stats pieces.)
83
84     ### 4b. Tiered model (SEALED-FIXTURES)
85     | Tier | Where | Key? | Contents | Test behaviour |
86     |---|---|---|---|---|
87     | Tier-0 | `eval_baselines/fixtures`/ (tracked) | none | opaque surrogate slug,
fps/frame_width (retained, NOT quantized), the de-attributed per-frame `trajectory.csv`
(Frame,Visibility,X,Y) (shuttle-only, time-origin reset), the 5 shuttle feature columns inside
`expected.json`, strict-slug `label`, relative `gts`. No person/audio/pose ? NOT minimized
to 5 scalars ? see §4c. | Unconditional ? runs on every PR/fork; turns today's skipped
litmus into a real public gate |
88     | Tier-1 | OneDrive (default) or SOPS-encrypted in-repo | token/age key | full
corpus with person/audio; authoritative ? F1/CI/sign-test + exact Gate-A litmus | Skips
cleanly when key/data absent (generalize `_golden_present()` ? `_tier1_present()` ) |
89
90     A fresh no-key clone is always green; coverage never depends on a secret.
91
92     ### 4c. Anti-attribution measures (Tier-0)
93
94     > ? RISK-ACCEPTANCE RE-CONFIRMATION REQUIRED (audit 2026-06-16). An earlier draft of
this doc claimed Tier-0 commits "only ~5 aggregate shuttle scalars" and never the per-frame
CSV". That is not what ships. The shipped freeze (`freeze_real_fixture`) commits the FULL
per-frame `(Frame,Visibility,X,Y)` `trajectory.csv` ? de-attributed (time-origin reset +
opaque random slug + metadata strip), but not minimized to 5 scalars. The per-frame stream
is the de Montjoye fingerprint (4 points ? 95% re-ID against a published reference). The owner's
2026-06-16 risk acceptance (§7 P2) was recorded against the inaccurate "5 scalars" basis and
MUST be RE-CONFIRMED on this corrected basis before any real Tier-0 fixture is committed
(P3). Synthetic fixtures (current state) carry zero provenance risk regardless.
95
96     - Opaque RANDOM surrogate id (`os.urandom` + a private surrogate?source map kept
off-git) ? not the MD5 `video_id`, and not HMAC(salt, video_id?name) (red-team: brute-
forceable over a tiny known roster if the salt leaks). Never persist the source `name`/filename.
97     - Strict-slug `label` + dropped `source_note` ? the operator `label` is validated
against `[a-z0-9][a-z0-9-]*` (a free-text identifier like a player/event name is refused),
and operator free-text `source_note` is accepted-but-NOT-persisted (never reaches the
committed `provenance.json`) ? both close the free-text identifier leak `_scan_forbidden` could
not police (audit fix #1).
98     - Metadata strip ? no filename, video_id, match/event/player identity, capture date.
99     - Time-origin reset ? subtract per-fixture `t0` (6dp-rounded) from all
start/end/gts. Metric-invariant within `_FLOAT_TOL` (`metrics.interval_iou` and start/end-
error are pure differences): the genuine (raw) reset-vs-no-reset score delta sits at the float-
error floor (~3.3e-7); bit-identical for whole-second offsets. See §4c-note below and §6.
100    - Per-frame trajectory IS committed (de-attributed), NOT minimized to scalars ?
Tier-0 commits the de-attributed per-frame `trajectory.csv` (shuttle stream only). It is not
reduced to ~5 aggregate scalars; the 5 shuttle feature columns appear (computed) inside
`expected.json`, but the raw per-frame `(Frame,Visibility,X,Y)` stream is part of the committed
fixture. (This is the corrected basis the ? note above gates on.)
101    - Stream minimization (streams, not resolution) ? Tier-0 keeps the shuttle block
only; person/audio excluded; pose/gait/face have no field in the schema by design (positive
`_scan_forbidden` assertion on the written bytes).
102    - fps/frame_width are RETAINED as scale constants ? they are NOT quantized today

```

(quantization is a deferred P3 option, see §7 ?). Reconcile any "quantize fps/frame_width" wording elsewhere to this.

103 - ****Provenance hard-fail gate**** ? the de-attribution pass refuses to emit a public fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + non-blank-license, and refuses a negative reset label (timeline predates the trajectory). Writes go to a temp dir and are promoted only after the privacy scan passes, so a refused freeze leaves no partial fixture (audit fixes #3a/#3b).

104

105 **### 4d. The two gates**

106 - ****Characterization/snapshot**** ? replay fixture, ****parse-compare**** JSON (never byte-compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields, `approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints (incommensurable ? loud fail). Catches any behaviour change even at constant F1. ****CONTROL** path only** across machines.

107 - ****Quality-floor**** ? score vs committed labels; assert per-video + mean F1 ? floor in `eval\_baselines/regression\_baseline.json` (created here; carries a ****windowing fingerprint**** so a served-default change is flagged incommensurable, not silently compared); emit `eval\_stats` paired-delta-CI + sign test (NUMBERS INFORM); add a ****metrics-vs-base-branch**** paired delta. Synthetic-tier floors measure ****correctness**, not field quality** (documented).

108

109 **### 4e. Serialization & optional protobuf**

110 JSON with `sort\_keys=True` + pre-quantized floats + LF is sufficient. ****Protobuf** is optional (01)** ? adopt only if cross-OS byte-stable snapshots are wanted (proto3 `rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `\*.pb binary -text`). Protobuf is ****encoding**, not ****secrecy**** (trivially `--decode\_raw`).

111

112 **### 4f. Encryption & keys (Tier-1 only)**

113 ****Default = out-of-repo + token**** (existing OneDrive `RALLY\_GOLDEN\_DIR` seam ? zero sensitive bytes in git history, sidesteps the legal "redistribution" act). ****Alternative = SOPS + age**** (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI secret exposed ****only** to the Tier-1 job on trusted branches, never to fork-PR runners**. The ****HMAC/surrogate salt**** (if used) and the surrogate?source map live with the private corpus, rotated per release. Document explicitly that Tier-1 crypto defends ****only**** the no-key public.

114

115 ---

116

117 **## 5. Threat model (who's protected vs not)**

118

119 | Actor | Protected against | NOT protected (by design / residual) |

120 |---|---|---|

121 | ****TM1 no-key public**** | map fixture?source/match/player; decrypt Tier-1; confirm a source even holding the candidate file | read Tier-0 numbers + run shuttle-only harness (intended; useless-in-isolation) |

122 | ****TM2 keyed contributor**** | recover filename/video_id/match/face/pose/gait ? *never written* (survives the analog hole) | read every value the suite reads (unpreventable; not pretended otherwise) |

123 | ****TM3 CI runner**** | leaked log can't leak attribution it never had | runs the suite (intended); Tier-1 secret never on fork runners |

124 | ****TM4 malicious keyed insider**** | correlation needs a public reference; Fork-A source is unpublished ? "not reasonably likely" | ****residual:**** conditional + revocable ? publish/leak the source later and the link can snap back (forward-looking re-test duty) |

125 | ****TM5 re-ID adversary**** | source unpublished + reset timeline ? no back-match against a public reference | ? becomes live if a Fork-A source is ever published ? ****and** the committed per-frame trajectory IS a fingerprint** (de Montjoye 4-point re-ID), NOT just ~5 scalars (corrected §4c). The protection is the unpublished source, not minimization |

126 | ****TM6 accidental plaintext commit**** | required CI leak-scan rejects MD5/known-name/non-surrogate/plaintext-SOPS | if the required check is disabled, a leak lands |

127

128 ---

129

130 **## 6. Honest limits ? what a green suite does NOT prove**

131

132 - Green proves only that the ****deterministic post-trajectory transforms**** are unchanged for the ****frozen cases****. It does ****not**** cover: the WASB/TrackNet detector,

```

`_probe_fps_width`/cv2 frame-seek, the AI/provider handoff, chunking/re-encode, the ffmpeg
stitch, DB persistence/migrations, or the API filter. *A refactor that breaks decoding or the
detector passes these gates green.* The owner-box `regression.py` DB run remains the only full-
chain check.
133   - Characterization = **behaviour-locking, not correctness** ? it freezes current
behaviour *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour
correct". This caveat must ride in the **test pass/fail message**, not only the docs.
134   - **Synthetic-tier floors = plumbing correctness, NOT field quality.** Never cite
synthetic F1 in `QUALITY_ITERATIONS.md` / the paper as rally-detection performance (tag `kind:
synthetic`).
135   - Tier-0 public coverage is **shuttle-only** ; the person/audio fusion ?F1 and exact
Gate-A values need Tier-1 (skipped on no-key clones).
136   - Data is **not "anonymous"*** absolutely (EDPS v. SRB) ? relative, conditional,
revocable, forward-looking.
137   - The committed Tier-0 stream is the **de-attributed per-frame `trajectory.csv`**, NOT
~5 scalars ? a per-frame trajectory is a de Montjoye fingerprint; the only protection is the
**unpublished source** (§4c ?, §5 TM5).
138   - **Time-origin reset is metric-invariant within `_FLOAT_TOL`, not byte-exact for every
offset.** The genuine (raw) reset-vs-no-reset score delta is at the float-error floor (~3.3e-7);
it is **bit-identical for whole-second offsets**, but because 6dp-rounding is not additive
(`round6(a)-round6(b) ? round6(a-b)`) the *serialized* 6dp boundary error can differ by one 6dp
ULP (1e-6) on a sub-second offset ? the serialization grid, not a metric divergence. (Audit fix
#2 rounds `t0`+labels to 6dp before the shift to keep the residual ? the grid step.)
139   - Encryption gives **no secrecy from a running contributor** ; protobuf gives **no
secrecy at all**.
140   - De-attribution does **not** cure provenance/licensing ? it sits *behind* the per-
record provenance gate + counsel sign-off.
141   - `--refresh-snapshot` can rubber-stamp a real regression; mitigation is **review of the
visible diff** + a separate, deliberate floor-baseline update. **A recorded `--reason` is
REQUIRED** (the CLI errors without one ? audit fix #6).
142
143   ---
144
145   ## 7. Deliverables & progress tracker ? **source of truth**
146
147   Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One **small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
148
149   | ID | Deliverable | Depends on | Counsel-gated? | Status | PR |
150   |----|-----|-----|-----|-----|----|

```

33. assistant (2026-06-22T09:13:05.613Z)

Let me search for references to actual golden video files or the manifest that the nightly script needs:

34. user (2026-06-22T09:13:08.506Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/past_projects/DECOUPLED_COMPUTE/01-EXISTING-
PLANS-AND-GAP.md: CPU dev container) can run them,"* with `RALLY_GOLDEN_DIR` path indirection
and a **committed video-free
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/data_pipeline/GOLDEN_DATA_SHARING.md:- A
`RALLY_GOLDEN_DIR` env var (default `output/`) honored by `fusion_golden` / `fusion_compare` /
`ablation` so neither machine hardcodes the path. (The CLI `--out-dir` / `--features-dir` flags
already make this a convention; the env var is just ergonomics.)
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/GOLDEN_REGRESSION_FIXTURES.md:**Default = out-of-repo
+ token** (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero sensitive bytes in git history,
sidesteps the legal "redistribution" act). **Alternative = SOPS + age** (per-recipient,
revocable, value-level diffs) for Fork-A-clean-but-private data wanting in-repo versioning.
Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI secret exposed **only
to the Tier-1 job on trusted branches, never to fork-PR runners**. The **HMAC/surrogate salt**
(if used) and the surrogate?source map live with the private corpus, rotated per release.

```

Document explicitly that Tier-1 crypto defends **only** the no-key public.

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/GOLDEN_REGRESSION_FIXTURES.md:| 02 | *(optional)*
Tier-1 key-gated path. `scripts/fetch_golden.py` (OneDrive token, default) and/or SOPS+age;
`_tier1_present()`; `RALLY_GOLDEN_DIR` wired; Tier-1 CI job gated on secret, never on fork
runners. | M1 | No | ? | ? |
```

35. user (2026-06-22T09:13:08.512Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/fusion_audio.py:    python -m
backend.eval.fusion_audio --manifest output/human_lovo_manifest.json --features-dir output
--augment --compare
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/fusion_audio.py:    ap.add_argument("--
manifest", default="output/human_lovo_manifest.json")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/fusion_golden.py:    python -m
backend.eval.fusion_golden --manifest output/human_lovo_manifest.json --out-dir output
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/fusion_golden.py:    ap.add_argument("--
manifest", default="output/human_lovo_manifest.json")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/rally_seg_eval.py:GROUND TRUTH: the golden
manifest (`output/human_lovo_manifest.json`) gives each video's
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/rally_seg_eval.py:DEFAULT_MANIFEST =
os.path.join("output", "human_lovo_manifest.json")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/rally_seg_eval.py:#:
(`output/human_lovo_manifest.json`) is gitignored, so this committed set is the AUDITABLE
source
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/rally_seq_proto.py:    python -m
backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/served_gate_a.py:    python -m
backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/served_gate_a.py:    ap.add_argument("--
manifest", default="output/human_lovo_manifest.json")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/serve_contrast.py:    python -m
backend.eval.serve_contrast --manifest output/human_lovo_manifest.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/eval/serve_contrast.py:    ap.add_argument("--
manifest", default="output/human_lovo_manifest.json",
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/CODE_AUDIT_AND_TEST_HARDENING-
COMPLETED-2026-06-14.md:- One or two videos with high-quality golden labels (the ones used for
regression baselines or `human_lovo_manifest.json` style).
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/quality-iterations/2026-06-18-lovo-
resweep-n13/lovo_resweep.py:MANIFEST = os.path.join(MAIN, "output", "human_lovo_manifest.json")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/quality-iterations/2026-06-first-ab-
experiment/README.md: (`output/human_lovo_manifest.json`); detection was never re-run.
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/quality-iterations/2026-06-multivideo-
lovo/README.md:GPU never double-booked, race-free via a `kushagra_traj.csv`-exists gate).
Manifest: `output/human_lovo_manifest.json`.
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/quality-iterations/2026-06-multivideo-
lovo/README.md:`output/largetestvideo_vs_human.csv`. Manifest now n=3
(`output/human_lovo_manifest.json`).
```

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/quality-iterations/2026-06-multivideo-
lovo/README.md:python -m backend.eval.calibrate_local --manifest
output/human_lovo_manifest.json --metric f1 # heuristic LOVO
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/archives/quality-iterations/2026-06-multivideo-
lovo/README.md:python -m backend.eval.rally_seq_proto --manifest
output/human_lovo_manifest.json # learned prototype
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/docs/CODE_MAP.md:- `@scripts/nightly_regression.py::main`
? nightly rally-quality tripwire (the dual-eval-set discipline made automatic). Re-scores
the golden corpus (`output/human_lovo_manifest.json`) under TWO configs ? `default` (SERVED,
milestone knobs OFF) + `milestone` (R3 cap=6 + R4 inactivity trim) ? via
`backend.eval.rally_seq_eval.evaluate` (CPU, seconds; CACHED trajectories, no
GPU/Gemini/WASB**), diffs headline metrics (`tol_f1`, tIoU `f1@0.5/0.25`, `merge_rate`) + the
per-video over-seg guard (`split_count`/`merge_count` may not rise on ANY video) vs frozen
`eval_baselines/nightly_baseline.json`. Pure core
(`summarize_run`/`compare_config`/`compare_all`/`format_report`) is unit-tested; thin I/O shell
scores + writes `output/nightly_regression/<date>.{md,json}`. Exit 0=pass, 1=regression,
2=missing corpus, 3=no baseline, 4=stale (corpus/config changed ? re-snapshot)**. `--update-
baseline` snapshots HEAD. Default-ADDITIVE ? does NOT change the promotion gate. Wired as a
local Windows Scheduled Task via `scripts/nightly_regression.ps1` (wrapper) +
`scripts/register_nightly_task.ps1` (a cloud agent can't ? the cached trajectories live outside
the repo). The future RAW eval set (currently-bad videos) plugs in as a third config. See
`docs/R2_EVAL_METRIC_DESIGN.md` + `memory/eval-dual-set-regression`.

```

36. assistant (2026-06-22T09:13:15.859Z)

Now let me check the current state of the repository to see if there are committed fixtures or if the manifest/golden features are missing:

37. user (2026-06-22T09:13:16.103Z)

```

1      """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2      over-segmentation-sensitive metric set, honestly.
3
4      This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` $4.5 /
$7)
5      measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6      IoU-F1 is blind to merged mega-windows, so we report it alongside segmental F1@k,
the
7      segment-count ratio, and the explicit `merge_split` breakdown
(`segmentation_metrics`).
8
9      For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10     windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11     operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12     the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13     diff two windowings with a paired sign-test + CI (the per-tier `delta`).
14
15     GROUND TRUTH: the golden manifest (`output/human_lovo_manifest.json`) gives each
video's
16     trajectory path + fps + frame_width; the GT rally intervals come from
17     `output/<name>_golden_features.json` (the `gts` key) so this needs no extra label
files.
18
19     Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20     """
21     from __future__ import annotations

```

```

22
23     import argparse
24     import json
25     import os
26     from typing import Any, Dict, List, Optional, Tuple
27
28     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29     from backend.eval.metrics import Interval
30     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31
32     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
33     DEFAULT_FEATURES_DIR = "output"
34     DEFAULT_IOU = 0.5
35
36     #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
37     #: (end ? start) exceeds this. The local detector over-fires, and its false positives
are mostly
38     #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
long rallies
39     #: strips most of those FPs and reveals the real product quality on the long-rally happy
path. Used
40     #: as the default split point for the stratified report (`--long-tau` overrides it).
DURATION, not
41     #: shot-count, is the filter: golden `shots_count` is unlabeled and the no-AI path
reports it as
42     #: "Unknown", whereas duration is derived from the same start/end labels we already
trust.
43     LONG_RALLY_TAU = 5.0
44
45     #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
manifest
46     #: (`output/human_lovo_manifest.json`) is gitignored, so this committed set is the
AUDITABLE source
47     #: of the single-vs-multicourt strata, kept in sync with `docs/GOLDEN_VIDEOS.md` (the
clips tagged
48     #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
49     #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
50     #: explicit `court_type` field, which overrides this set (see :func:`court_type_for`).
51     MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2",
"Boxhill_Doubles"})
52
53
54     def court_type_for(video: Dict[str, Any]) -> str:
55         """`multicourt` or `single` for a golden video. Honors an explicit manifest
56         `court_type` field when present, else falls back to membership in
`MULTICOURT_CLIPS`
57         (keyed by `name`), else `single`."""
58         ct = video.get("court_type")
59         if isinstance(ct, str) and ct:
60             return ct
61         return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
62
63
64     def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
65         """The long-rally lens: keep only intervals strictly longer than `min_duration`
seconds.
66
67         `min_duration <= 0` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
68         positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
69         to predictions AND ground truth before any matching, so the tIoU and R2 paths stay

```

```

consistent."""
70     if min_duration <= 0.0:
71         return list(ivs)
72     return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
73
74
75     # ----- #
76     # Pure scoring core (no I/O ? unit-testable)
77     # ----- #
78     def score_intervals(preds: List[Interval], gts: List[Interval],
79                         iou: float = DEFAULT_IOU,
80                         tau_start: Optional[float] = None,
81                         tau_end: Optional[float] = None,
82                         min_duration: float = 0.0) -> Dict[str, Any]:
83         """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85         Pairs temporal-IoU P/R/F1 + boundary error (`metrics.score`) with the
86         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
87         so a
88         merged mega-window is *visible* even when IoU-F1 isn't moved.
89
90         Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline
91         going forward;
92         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
93         strict-1:1
94         tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs
95         precision) + the
96         start/end boundary-error medians + the over-seg guard counts. See
97         docs/R2_EVAL_METRIC_DESIGN.md.
98
99         ``min_duration`` > 0 applies the **long-rally lens**
100         (:func:`filter_by_min_duration`): BOTH preds
101         and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
102         so the whole
103         row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
104         passthrough).
105         """
106         from backend.eval import tolerance_metrics as tm
107         preds = filter_by_min_duration(preds, min_duration)
108         gts = filter_by_min_duration(gts, min_duration)
109         ts = tm.TAU_START if tau_start is None else tau_start
110         te = tm.TAU_END if tau_end is None else tau_end
111         sc = metrics.score(preds, gts, iou_threshold=iou)
112         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
113         ms = segmentation_metrics.merge_split_report(preds, gts)
114         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
115         be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
116         R4-aiming diagnostic)
117         osr = tm.over_seg_report(preds, gts)
118         return {
119             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
120             sc.mean_iou,
121             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
122             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
123             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
124             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
125             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
126             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
127             # --- R2 (tolerance-match) ? the task-faithful headline block ---
128             "tau_start": ts, "tau_end": te,
129             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
130             "tol_matched": float(len(tol_matches)),

```

Now let me look at whether the claim's preconditions (golden videos, cached WASB results) actually exist locally:

39. user (2026-06-22T09:13:21.425Z)

```
find: 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/output': No such file or directory
```

40. user (2026-06-22T09:13:22.008Z)

(Bash completed with no output)

41. assistant (2026-06-22T09:13:24.022Z)

Let me check if this is documented and check the issue status:

42. user (2026-06-22T09:13:26.136Z)

```
Found 4 files
.claude\worktrees\nightly-e2e\docs\QUALITY_ITERATIONS.md
.claude\worktrees\nightly-e2e\docs\PROMINENT_COURT_DETECTION.md
.claude\worktrees\nightly-e2e\docs\GOLDEN_REGRESSION_FIXTURES.md
.claude\worktrees\nightly-e2e\docs\CODE_MAP.md
```

43. assistant (2026-06-22T09:13:28.175Z)

Let me check the .gitignore to understand what's excluded:

44. user (2026-06-22T09:13:28.819Z)

```
1      # Byte-compiled / optimized / DLL files
2      __pycache__/
3      *.py[cod]
4      *$py.class
5      *.pt
6
7      # Local environment secrets
8      .env
9      *.env
10     .env.*
11
12     # Provenance stamp baked into a deploy tarball at build time (deploy/gcp/README.md §4);
13     # never a tracked source file ? generated per-deploy, must not ride into a commit.
14     /GIT_SHA
15
16     # C extensions
17     *.so
18
19     # Distribution / packaging
20     .Python
21     build/
22     develop-eggs/
23     dist/
24     downloads/
25     eggs/
26     .eggs/
27     lib/
28     lib64/
29     parts/
30     sdist/
31     var/
32     wheels/
33     share/python-wheels/
34     *.egg-info/
```

```
35     .installed.cfg
36     *.egg
37     MANIFEST
38
39     # SQLite databases & Local output files
40     *.db
41     *.db-journal
42     *.db-wal
43     output/
44     output_dir/
45     backend/static/highlights_*.mp4
46     backend/static/*_compiled.mp4
47
48     # OS generated files
49     .DS_Store
50     Thumbs.db
51     desktop.ini
52
53     # Coverage data
54     .coverage
55     .coverage.*
56     htmlcov/
57     .pytest_cache/
58
59     # Local Claude Code config (machine-specific)
60     .claude/
61     .coveage
62     .
63     .
64     # Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
65     # ruff + pytest; never meant to be committed ? keep them out of `git status` too).
66     scratch/
67     .venv/
68
```

45. assistant (2026-06-22T09:13:32.124Z)

Perfect. Now let me verify the final claim against the actual evidence found:

46. user (2026-06-22T09:13:54.066Z)

Structured output provided successfully